RHO: Your Coding Agent is Secretly a Roboticist

Karim Elmaaroufi
University of California, Berkeley
Embodied Science

Justin Svegliato
University of California, Berkeley

Sarunas Kalade
AMD

Graham Schelle
AMD

Sanjit A. Seshia
University of California, Berkeley

Matei Zaharia
University of California, Berkeley

Abstract: Code-as-Policies (CaP) has shown that large language models (LLMs) can write code to solve robotics tasks by composing perception, planning, and control primitives. Recent CaP systems, however, rely on multi-turn code-generation loops at test time, which is often infeasible for real-time robot control. We introduce **Robotics Harness Optimization (RHO)**, a novel paradigm in which tool-enabled coding agents, *at training time*, propose and search for interpretable, neurosymbolic multi-file policy repositories (*Repositories-as-Policies*) that compose these primitives rather than a single prompt, function, or file. RHO searches with reflective feedback from environment reward and execution rather than teleoperation demonstrations. It generalizes to perturbed pick-and-place settings like LIBERO-PRO, where OpenVLA scores 0.0% and $\pi_{0.5}$ averages 12.83%. Using the same low-level primitives, RHO reaches a 45.0% success rate, 2.5$\times$ higher than the strongest multi-turn agentic system, and 3.5$\times$ higher than $\pi_{0.5}$. On Robosuite, RHO sets a new state-of-the-art of 70.0%, exceeding the prior multi-turn record of 68.29% using single-turn execution with no corrective LLM code edits at deployment. When an LLM is used in the control loop, as on RAI's O3DE benchmark, RHO optimizes the deployed agent's multi-file harness of prompts, tools, and control code, improving held-out success from 23.5% to 44.3% with 20% less wall-clock time and 27% fewer tool calls.

1 Introduction

Code-as-Policies (CaP) has opened new possibilities for robot control by enabling large language models (LLMs) to write executable robot programs. Rather than representing policies as black-box neural networks, CaP enables interpretability with LLMs that write code to compose perception, planning, and action primitives. These programs enable robots to perform tasks such as stacking blocks [1], washing plates [2], and setting dining tables [3]. Recent CaP systems use test-time methods including multi-turn rounds, ensembles of candidate code completions, and visual-difference modules that convert execution feedback into prompt context [4]. While effective, these methods rely on iterative code generation loops at test time, often leaving them unsuitable for real-time robotics tasks. We show that scaling LLM test-time inference at deployment is unnecessary.

CaP agents and vision-language-action (VLA) models each fail differently, and current evaluations document both [4, 5]. The best-performing agent on CaP-Bench (CaP-Agent0) issues up to 66 LLM calls at test time per trial. Yet scaling inference calls is not reliably beneficial [6]. Restricting the same agentic system to a single turn without that scaffolding makes reported Robosuite success collapse from **68.29%** to **24%** [4]. Meanwhile, VLAs struggle to generalize: models like OpenVLA and $\pi_{0.5}$ [7, 8] that were trained on LIBERO demonstrations [9] collapse to **0.0%** and **12.83%**, respectively [5]. Other perturbation suites report related VLA generalization failures [10, 11, 12, 13].

We introduce **Robotics Harness Optimization (RHO)**, a novel paradigm for evolving interpretable neurosymbolic multi-file robot-policy repositories (*Repositories-as-Policies*). To instantiate RHO,



Figure 1: RHO system overview. A parent repository sampled from a Pareto frontier is mutated by a tool-enabled coding agent that edits multiple files and executes candidate policies in an isolated workspace with whitelisted access to a robot environment. If the child passes the configured training gate, it is evaluated on all validation examples and added to the Pareto frontier. At the end of evolution, the frontier repository with the highest mean validation reward is deployed as the policy.

we generalize GEPA (Genetic-Pareto), a reflective optimizer that mutates artifacts through natural-language reflection while keeping a Pareto frontier of candidates [14], and Optimize Anything [15], into HELIX (Hierarchical Evolution via LLM-Informed eXploration). *As our instantiation of RHO, HELIX is the first reflective optimizer that (1) lifts the optimization surface to whole multi-file repositories rather than a single prompt or function, and (2) uses a tool-enabled coding agent as a mutator that, within one mutation step, can write ephemeral tests, run candidate code in the robot environment, read tracebacks, query a visual-difference tool, and edit multiple files.* We enable live interaction with the robot environment by designing a robotics-environment-as-a-service (REaaS), and finally, thoughtful isolation mechanisms between the coding agent, REaaS, and the evaluator to prevent cheating and reward hacking. As a result, each generation retains useful repositories on a per-instance Pareto/coverage frontier. The final artifact is a multi-file neurosymbolic repository that establishes several state-of-the-art results and, unlike VLAs, is highly interpretable and debuggable.

On the CaP-Bench Robosuite [16] manipulation benchmark, RHO reaches **70.0%** (490 of 700 trials) under single-turn evaluations, exceeding the strongest published record (68.29%), which required multi-turn code generation and visual-difference feedback at test time [4]. RHO also exceeds prior results when LLMs remain part of the control loop: on Robotec's RAI ROS 2 manipulation benchmark (O3DE) [17], RHO evolves an entire multi-file repository of prompts, tools, and control code, nearly doubling held-out success from **23.5%** to **44.3%** with **20%** less wall-clock time and **27%** fewer tool calls. On the six LIBERO-PRO [5] perturbation cells reported in CaP-Bench, where VLAs fail to generalize (OpenVLA: 0.0% and $\pi_{0.5}$: 12.83%), RHO trains by interacting in the LIBERO environment (absent of demonstrations) and, within 63 generations, records a new state-of-the-art for CaP methods of **45.0%** (1351 of 3000 trials) under single-turn evaluations, a 3.5x improvement over $\pi_{0.5}$ and a 2.5x improvement over the strongest prior method which required multi-turn corrective actions (CaP-Agent0 at 18.17%). In summary, our contributions are:

1. **Robotics Harness Optimization.** RHO, a novel paradigm that shifts the computational burden from test-time code generation to *training time* via a new reflective evolutionary optimization framework for robotics, whose optimized artifact is a multi-file policy repository rather than a single prompt or function, yielding interpretable neurosymbolic policies.
2. **Tool-enabled reflective mutation.** The first evolutionary reflective optimizer whose mutator is a tool-enabled coding agent that can execute tests, run code against the REaaS, read tracebacks, query visual-difference feedback, and edit multiple files within a generation.

3. **Robust policies that beat agentic stacks and VLAs.** In single-turn evaluations, RHO reaches **70.0%** on CaP-Bench Robosuite (versus the multi-turn record of **68.29%**) with zero LLM code-generation calls in the control loop, nearly doubles held-out RAI O3DE success from **23.5%** to **44.3%** with **20%** less wall-clock time and **27%** fewer tool calls, and reaches **45.0%** on LIBERO-PRO (versus prior CaP SOTA of **18.17%**) where OpenVLA and $\pi_{0.5}$ average 0.0% and 12.83%, respectively.

2 Related Work

Learned visuomotor policies. End-to-end VLAs map pixels and language to low-level actions within a single trained network rather than a separate planner [18, 19, 7, 20, 21, 8, 22, 23, 24, 25, 26]; world-action variants add observation prediction or visual-goal generation [27, 28, 29], and model-based alternatives train a latent world model for explicit optimization or imagined rollouts [30, 31, 32, 33]. Robustness across these paradigms remains bounded by the training distribution: independent audits report tens-of-points collapses under viewpoint, paraphrase, and layout perturbations [10, 5, 12], and observation-predictive models produce visually plausible but physically inconsistent rollouts [34]. Unlike these distribution-bound paradigms, RHO recovers the modular structure that lets task-and-motion planning generalize across novel goals and object arrangements [35], using a coding agent under evolutionary search to compose a multi-file repository over the same perception and control primitives.

Code-as-Policies and language-driven robot agents. CaP recasts the control policy as code an LLM writes from a task description, with primitives and feedback supplied as in-context examples [1, 2, 36, 37, 38, 3]. Planner-with-skill-library systems instead keep the model frozen and call it repeatedly to sequence a fixed library of pretrained skills, grounding each choice through affordance value functions or iterative visual prompting and replanning when a step fails [39, 40, 41, 42, 43]. Unlike CaP, the model selects among existing skills rather than writing the control code itself. Deployment-facing descendants instead keep an LLM in the control loop at test time, re-invoking it between executions to attribute a failure, replan, or synthesize and hot-reload a revised controller program [44, 45, 46]. Where the planners above consult a frozen VLM that never edits the policy, these agents rewrite the executing policy itself, and scale the number of such calls per step. The nearest such system is CaP-Agent0 [4], a multi-turn tool-enabled agent that issues up to eleven LLM calls per turn for up to six turns. RHO instead front-loads LLM code generation and corrective editing into training-time evolutionary search.

LLM optimization loops in robotics. Reward-design and curriculum-generation systems place the LLM upstream of a learner: the LLM authors the supervision signal while a separate trainer produces the policy. The closest is Eureka, which uses an LLM to author and refine dense reward functions for RL against rollout statistics [47]. Across these systems, the LLM writes the objective, while reinforcement learning trains a neural network policy: the LLM synthesizes rewards and curricula [48, 49, 50, 51, 52, 53, 54, 55, 56], grounds those rewards in VLM critics [57, 58], generates the training environments themselves [59, 60, 61, 62, 63, 64], structures the reward search with graph-of-thoughts or multi-agent shaping [65, 66], and extends to locomotion, navigation, and morphology co-design [67, 68, 69]. RHO differs by evolving the policy artifact directly: no reward function to shape, no curriculum, no neural policy trained on the LLM output.

Optimizing agents. Optimizing the prompts, memory, retrieval, and tool calls that surround a model, rather than its weights, is a nascent direction [70, 71, 72], and to date sits almost entirely in text domains where rewards are cheap to verify, such as math, where rule-checkable answers have driven reinforcement-learning progress [73]. The most direct comparison is Meta-Harness [74], which searches over single-file programs that implement prompting, retrieval, and orchestration logic across text-classification, math, and coding benchmarks such as TerminalBench-2 [75]. Recursive Language Models instead recurse over the prompt at inference to reason over long contexts [76], though a reproduction shows the benefit does not compound: recursing more deeply degrades ac-

curacy while inflating latency nearly a hundredfold [77]. RHO differs by grounding candidates in live robot-environment interaction rather than a static text benchmark, and by amortizing the coding agent's interactions into training-time search for multi-file solutions rather than single file solutions.

Reflective evolution. Reflective evolution treats the LLM as a search operator over user-defined evaluator-scored text artifacts: FunSearch searches program snippets for combinatorial constructions [78], AlphaEvolve broadens it to scientific discovery [79], and GEPA to prompt optimization [14]. Verbal self-critique and code refinement loops over agent trajectories preceded this line [80, 81]; recent extensions evolve coding agents themselves [82, 83, 84] or scale deploy-time deliberation through structured inference [85, 86, 87]. Most closely related is `optimize_anything` [15], whose reflective text-artifact evolution recipe our optimizer HELIX inherits. To our knowledge, HELIX is the first reflective evolutionary optimizer whose mutator is a tool-enabled coding agent and whose candidate is a whole multi-file application repository evolved against a live environment, rather than a single text artifact or the search agent's own code [82]. We are also the first to bring reflective evolutionary optimization to the learning of deployable robot-manipulation policies that are searched directly against live environment interaction rather than a static benchmark. Our initial attempts suffered from the coding agent's cheating and ultimately inspired RHO's isolation mechanisms between the evaluator, REaaS, and the coding agent. Other works have observed similar cheating in coding domains [88].

3 RHO: Searching over multi-file repositories-as-policies

Robotics Harness Optimization (RHO) is a novel paradigm for evolving interpretable neurosymbolic multi-file robot-policy repositories (i.e., *Repositories-as-Policies*). In this paper, we instantiate RHO by combining HELIX, robot-environments-as-a-service (REaaS), and isolation between the evaluator, REaaS, and mutation agents. HELIX mutates whole repositories with tool-enabled coding agents, REaaS exposes robot-environment rollouts during search, and isolation separates everything such that cheating (i.e., searching for answers in the benchmark) is not possible. A candidate repository contains everything needed to instantiate a policy: a task router, control routines, perception wrappers, grasp and pose helpers, and in LLM-in-the-loop deployments, the LLM's harness. Evolution returns one repository evolved from a seed (i.e., an empty directory or a user-provided skeleton), which is then deployed. The repository is neurosymbolic: symbolic code composes learned perception and grasping (neural-network-based) services with motion primitives. In CaP-Bench settings, the repository executes as a Python controller with no corrective code edits from an LLM at deployment. In Robotec's RAI ROS 2 stack, where the control loop includes an LLM, the repository instead defines the prompting, control interface, and tools used by that LLM-in-the-loop system.

RHO follows the reflective evolutionary optimization recipe of GEPA's `optimize_anything` [14, 15]: evaluate a candidate, summarize failures and successes as reflective feedback, mutate the candidate, and retain improvements on a frontier. The key enhancements are the mutation operator, search space, environment interface, and isolation boundary. Prior reflective optimizers typically expose a bounded artifact (a function, prompt, delimited code region, etc.) to an LLM completion API. HELIX exposes a whole repository to a tool-enabled coding agent. A mutation is therefore an interactive development session: the agent inspects files, edits multiple modules, writes and executes ephemeral tests, runs candidate policies through a whitelisted REaaS, reads tracebacks, queries visual-difference feedback, and revises the repository before submitting its candidate (Figure 1).

3.1 Repository-level mutation

Existing reflective evolutionary optimizers such as FunSearch [78], AlphaEvolve [79], GEPA [14] and `optimize_anything` [15] typically expose a single bounded artifact to the search operator. We instead expose a whole multi-file repository to the search operator, treating *Repositories-as-Policies*. In robotics, a manipulation failure may require changing how an object pose is estimated, how a motion primitive is parameterized, how retries are attempted, or how task-level language is

interpreted as geometric constraints, often all at once. A bounded-artifact mutator cannot reliably refactor across these interfaces in one move; it can only patch one of them and hope the others remain consistent. The current leading CaP-Bench agent handles this interleaving with multi-turn interactions and tens of LLM code-completion queries.

We formalize RHO's search problem as follows. Let $\mathcal{T}$ denote a set of robot-task instances. A repository $R \in \mathcal{R}$ defines an executable robot policy through its public entry points and supporting files. Running R on instance $\tau \in \mathcal{T}$ produces an evaluator output

$$E(R, \tau) \;=\; \big(\, s_\tau(R),\; z_\tau(R) \,\big),$$

where $s_\tau(R) \in [0, 1]$ is the environment-provided shaped reward returned by the task or benchmark and used for selection, and $z_\tau(R)$ is structured side information returned to the optimizer (Section 3.2). The optimizer never receives hidden simulator state or evaluator internals. RHO requires the user to supply the evaluator (a script that maps an instance to a per-instance score plus side information). In this paper, the evaluator runs each instance as a single $(\text{task}, \text{trial}, \text{seed})$ rollout of the candidate against the benchmark environment and returns the environment's final shaped reward as $s_\tau(R)$; the scalar objective RHO optimizes is then the mean of these per-instance rewards over the paired training minibatch (Eq. 1) and over the full validation set at terminal selection (Eq. 2).

The design choice of repository-level mutation makes the search variable a multi-file repository (Python in our CaP-Bench experiments; RHO's implementation, HELIX, imposes no language constraint, and the repository may mix files in any language the target runtime accepts) containing a task router, control routines, perception and primitive-call wrappers, grasp and pose helpers, and when the deployment loop includes an LLM, prompt scaffolds and tool-edit modules. A generation produces a child by applying a bounded coding-agent edit to a parent repository. This changes the unit of search from a bounded artifact like a prompt or file to a deployable software system.

The search couples two mechanisms: a scalar parent-to-child acceptance gate on a sampled training minibatch, and per-instance coverage-based frontier retention.

A candidate child R_c is accepted against its parent R_p when its total shaped reward over the paired training minibatch B strictly improves on the parent's:

$$\sum_{\tau \in B} s_\tau(R_c) \;>\; \sum_{\tau \in B} s_\tau(R_p), \tag{1}$$

with the strict inequality relaxed to $\geq$ in some experiments as it allows for even more candidate diversity. In practice, minibatches > 2 struggle to make progress as solving a single robotics task reliably is already challenging enough.

RHO adds two training-loop refinements over the parent algorithm: Stratified Batch Sampling across task families (sample tasks and then sample seeds within each task), and an optional Staged Validation gate that sits between minibatch acceptance and full validation; it is enabled in our LIBERO runs and disabled in the Robosuite canonical run. The frontier is maintained over N validation instances: each instance has its current best program, and a program is removed when every instance on which it was best is also covered by some other surviving program (GEPA-style coverage dominance), not by pairwise comparison against the parent. At the end of evolution the deployed repository is the highest-mean-validation-reward member of the terminal frontier F_T,

$$R^\star \;=\; \operatorname*{arg\,max}_{R \in F_T} \; \frac{1}{N} \sum_{i=1}^{N} s_i(R), \tag{2}$$

with ties broken by validation coverage. Further algorithm details are in GEPA [14].

Figure 2 illustrates why a Pareto frontier matters. Most lineages dead-end on candidates that were not further optimized; some are pruned (i.e., an agent previously made a poor decision and could not recover from it). We find that the deployed solver emerges from one of a few productive branches of the accepted-candidate tree. Thus, retaining a Pareto frontier rather than hill-climbing a single chain (i.e., a simple reflective loop) is what surfaces the deployed solver.

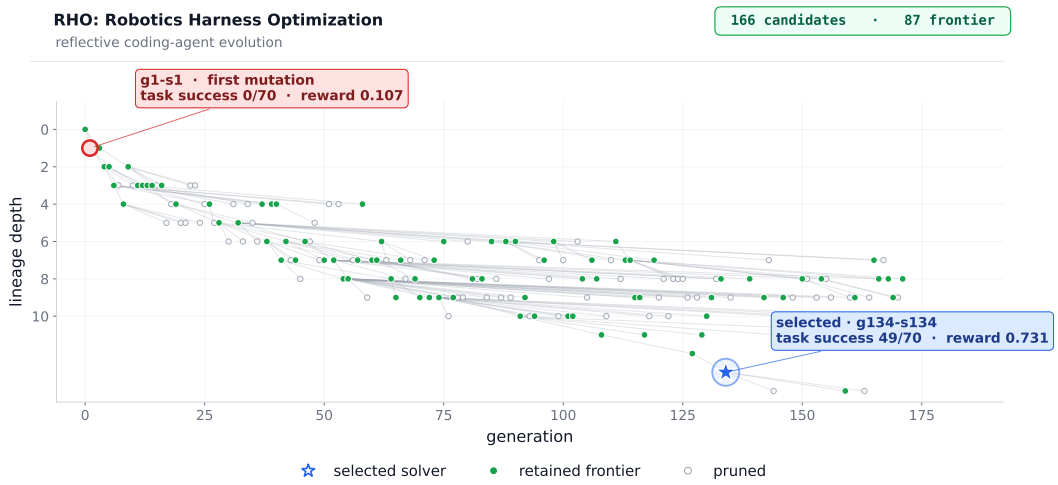


Figure 2: Why a Pareto frontier matters. The accepted-candidate lineage of the reported run splits into 87 repositories retained on the Pareto frontier and 79 pruned, with a further 77 rejected attempts and 28 skips never entering the lineage. Most branches dead-end, and the deployed solver emerges from one of a few productive branches that a greedy, single-chain search would have missed. Retaining candidates that are good at any task, but not necessarily all tasks (GEPA-style coverage dominance), is what provides the breadth to find the best solution.

3.2 Tool-Enabled Mutation

Reflective optimizers built on single-shot LLM completions emit a text patch in one response: the mutator cannot run a test, select and observe a desired traceback, inspect a rendered frame, or interact with the environment inside one genetic mutation step. For robotics, this constraint forces a generation of mutations to make many tightly coupled decisions blind to runtime state. A change that swaps a grasping primitive's reference frame, for instance, silently breaks a downstream verification check that the completion mutator has no way to discover before submission.

Tool-Enabled Mutation replaces the completion mutator with a tool-enabled coding agent (e.g., Codex, OpenCode) that, within one mutation step, can edit multiple files in a candidate repository, write and execute ephemeral tests, run candidate code against the REaaS, read tracebacks and `stderr`, and query a visual-difference tool that returns scene-difference text from candidate rollouts. Operationally, single-shot completion is a limiting case of this mutation operator in which the agent performs no intermediate tool calls. The agent is given the parent repository, the parent's full evaluator output for the current minibatch, and a reflection prompt that asks the agent to enhance the parent's strengths and address its weaknesses. The prompt template is in Appendix A.1.4.

Generation 0 is initialized from a coding-agent-authored seed repository that exposes the benchmark's invocation signature (e.g., `solver/main.py::run(ctx)` for CaP-Bench); the seed structure is an ablatable design choice whose configurations and candidates appear in Appendix A.1.5.

Mutation and evaluation surfaces. RHO deliberately separates the surface available during mutation from the surface used for scoring. The mutation surface follows CaP-Bench's M4 regime [4]: the coding agent may run candidate policies over low-level primitives, inspect `stdout` and `stderr`, query visual-difference feedback, and use primitive examples while debugging a child repository. However, our evaluation surface intentionally follows CaP-Bench's stricter S4 regime: the child is scored by executing its policy entry point in a single turn over the low-level primitive interface, without deployment-time VDM, multi-turn correction, or LLM code-generation calls. Any side information z produced by the evaluator is post-hoc: it is available only to future mutations, never to the repository during the scored rollout. Section 4 reports how strong agentic baselines collapse in the S4 regime when only their multi-turn correction and visual-difference options are removed.

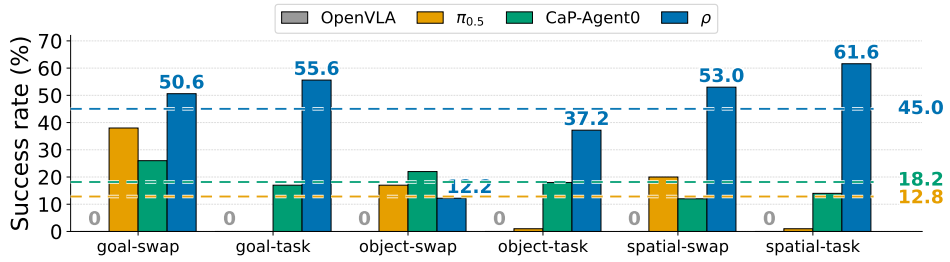


Figure 3: Three suites from LIBERO are composed with two types of perturbations: *swap* the initial positions of objects, and perturb *task* descriptions. CaP-Agent0 is reported under multi-turn results. Despite using only single-turn execution with no corrective code edits from an LLM at deployment, within 63 generations of optimization, RHO outperforms VLAs and the prior SOTA CaP system.

Integrity. Tool-enabled mutators with file-edit and shell privileges introduce a reward-hacking surface absent from completion mutators: a candidate may read evaluator internals or hardcode per-trial solutions, a failure mode documented in ML-engineering agents [88] and observed in our preliminary runs, where a mutator instructed to stop instead escalated to actions against the host environment itself (Section 7). RHO isolates mutation from scoring through a sidecar boundary that excludes evaluator source, benchmark splits, judge prompts, and benchmark metadata from the mutator workspace; the sidecar proxies whitelisted benchmark calls to host services, and a protected-file integrity check rejects children that alter evaluator-side artifacts. RHO uses a whitelisting approach and deploys it as a sidecar proxy container rather than containerizing all primitive services and robot-environments, as the latter incurs a much higher performance cost.

4 Experiments

We evaluate our RHO instantiation on three pick-and-place benchmarks with two frameworks supplying the perception and action primitives: LIBERO-PRO [5] and Robosuite [16] through CaP-Bench [4], and the O3DE manipulation benchmark through RAI [17]. Since a coding agent simply reading a file prior to editing would count as a turn, we expose the CaP-Bench M4 setting as our training regime, allowing multiple turns and access to a visual-difference module (VDM). For CaP-Bench and LIBERO-PRO, evaluation we intentionally restrict to the S4 setting: a single execution of the candidate repository, no VDM, and no LLM code-generation calls. RAI remains an LLM-in-the-loop deployment and is evaluated separately. Several research questions guide our experiments:

RQ1: How does RHO compare to current SOTA methods?
RQ2: Does RHO outperform coding agents?
RQ3: Are repositories the right unit of search, or does a different structural prior do better?
RQ4: Do solutions produced by RHO generalize?

Benchmarks and splits. We train two separate RHO instances on two CaP-Bench benchmarks [4] that share a primitive interface. The first consists of seven Robosuite manipulation tasks with 100 native trial IDs each (700 trials total). In all experiments, we train on the first ten trial IDs per task, leaving 90% unseen at evaluation. The second consists of three suites from LIBERO [9]: object, goal, and spatial. Each contains ten tasks with fifty initial states per task (1,500 instances total); training draws from 1,290 instances, the evaluator's full validation uses 210, and Staged Validation (Section 3.1) samples a 45-instance subset of that validation set for the intermediate gate, taken as the first 45 deterministic validation IDs (Appendix A.1.2). A subset of LIBERO-PRO [5] is the held-out perturbation extension that CaP-Bench uses for out-of-distribution evaluation. It composes (object, goal, spatial) $\times$ (swap, task) perturbations onto the three base LIBERO suites, yielding six cells of 500 trials each (3,000 trials total). The swap cells perturb the initial positions of objects; the task cells perturb the task description while keeping the scene fixed. RHO is not trained on LIBERO-PRO. Reported numbers come from the full evaluation sets under the CaP-X S4 surface.

Table 1: Per-task CaP-Bench + Robosuite success count at 100 trials/task. We intentionally restrict RHO to the hardest evaluation setting (single-turn S4-style) with no LLM code-generation calls in the control loop. CaP-Agent0 (up to 11 LLM calls per turn) reports its highest success under M4, which allows up to 6 turns and feedback from a VDM; S4 reports only the 24.00% aggregate.

Task	**RHO** (S4)	CaP-Agent0 (S4)	CaP-Agent0 (M4)
`cube_lifting`	98	—	97
`cube_stack`	98	—	98
`cube_restack`	54	—	89
`nut_assembly`	1	—	0
`spill_wipe`	100	—	100
`two_arm_lift`	53	—	74
`two_arm_handover`	86	—	20
Overall	**70.0**	24.00	68.29

Training. Our experiments use OpenAI Codex [89] and Claude Code [90] agents as backends. Each mutation step is capped at 150 agent turns per mutation unless otherwise specified. Robosuite runs use strict reward improvement on the paired minibatch, while LIBERO runs begin under the same rule but switch to admitting ties at the warm-restart that produces the reported solver.

Deployment CaP-Agent0's strongest results come from CaP-Bench's M4 evaluator setting: multi-turn execution with the visual-difference module enabled (11 LLM calls per turn, up to 6 times). Restricting to S4 for faster deployments (single-turn execution over the primitive API and no visual-difference module), CaP-Agent0's Robosuite score collapses from **68.29%** to **24.00%**. RHO trains under the M4 setting but reports results under the stricter S4 setting.

4.1 RQ1: How does RHO compare to current SOTA methods?

RHO exceeds the strongest published CaP-Bench agent on Robosuite and LIBERO-PRO and outperforms the reported VLA baselines on the perturbation extension. Both comparisons are conducted on held-out evaluation surfaces that differ from the training distribution.

CaP-Bench Robosuite. On the seven Robosuite manipulation tasks, our RHO instantiation reaches **70.0%** (490/700) under single-turn S4 deployment, slightly exceeding the strongest published multi-turn record from CaP-Agent0 (**68.29%** under M4 with up to 66 LLM calls per trial) while significantly reducing evaluation time [4]. The mutator sees only 10 of 100 trial IDs per task during training, so 90% of the evaluation surface is unseen.

CaP-Bench LIBERO-PRO. On the six LIBERO-PRO perturbation cells, our RHO instantiation reaches **45.0%** (1351/3000) after 63 generations of training on unperturbed LIBERO tasks without teleoperation demonstrations (Figure 3). This is a 2.5× margin over CaP-Agent0's **18.17%** (545/3000), a 3.5× margin over $\pi_{0.5}$'s **12.83%**, and exceeds OpenVLA's **0.0%** average [5, 7, 8].

4.2 RQ2: Does RHO outperform coding agents?

To determine if evolutionary loops are needed, we execute a single coding-agent mutation step with unlimited internal turns tasked with solving all seven Robosuite tasks. The agent operates in the M4 setting with access to its internal tools, the REaaS, and VDM. This experiment is repeated for different LLMs and both backends. We compare the results under S4 settings in Table 2.

All models and backends score between 13 and 30% on held-out S4, similar to CaP-Agent0's S4 result (24%) but far from CaP-Agent0's M4 result. Even with unlimited turns, we observe that agents will end their submissions prematurely far below the scores observed in multi-turn settings. Thus, reflective evolution is necessary to close the gap.

Table 2: Coding agents are given unlimited turns and a single mutation step of HELIX to solve all tasks in CaP-Bench Robosuite (trained under M4 but evaluated in S4). Single coding-agent mutations with full tool access reach 13–30%; CaP-Agent0 restricted to S4 lands at 24%. Token counts are taken from the per-mutation usage ledger that HELIX records for every coding-agent call, where permitted. VDM token usage is not tracked, as it is small relative to the mutator costs.

Configuration	input (M)	cached (M)	output (M)	cost (USD)	Robosuite S4
Codex GPT-5.5	4.31	4.20	0.018	$3.19	30.00%
Claude Code Opus 4.7	—	—	0.022	$1.46	16.43%
Claude Code Sonnet 4.6	—	—	0.039	$1.02	21.14%
Qwen3.6-27B (via Claude Code, local)	0.92	—	0.019	$0.00	13.43%
CaP-Agent0 (restricted to S4)	—	—	—	—	24.00%

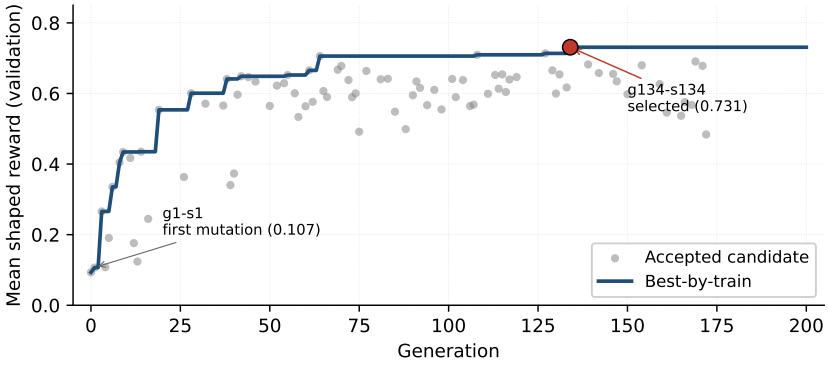


Figure 4: Reflective evolution leads to better solutions. Best-by-validation training reward across the multi-file experiment. In grey are the 87 accepted candidates at their promotion generation and in red is the highest reward candidate `g134-s134`. The selected repository improves mean shaped reward from 0.107 to 0.731 (+62.4 pp) and task success from 0/70 to 49/70; a single-shot Codex baseline reaches only 30.0% (210/700) against the deployed 70.0% (490/700).

4.3 RQ3: Is the repository the right unit of search?

Using Codex with GPT-5.5, we compare a multi-file repository seed against a single-file stub seed. Also, across models (Sonnet vs. Qwen3.6-27B) but the same backend, Claude Code, we compare solution representations (behavior trees vs. finite state machines) enforced through prompts.

With a smaller generation budget, the multi-file repository run at **70.0%** beats the single-file stub at **63.86%** despite using the same backend and tools (Table 3). It also sets a new Robosuite record under the S4 single-turn surface. Tighter priors collapse: Sonnet tasked with writing Behavior Trees reaches only **33.86%**, and Qwen with Finite State Machines reaches only **18.14%**. Qwen's free-form counterpart reaches 35.0%, a 16.86 pp lift from removing the FSM scaffold on the same backend (Appendix A.2.5). The repository sits at the right point of structural prior: enough scaffold to scope the agent toward modular code and not enough to constrain its solution space. The pattern is not Robosuite-specific: on Robotec's RAI O3DE manipulation benchmark, a GEPA-style single-file ablation increased tool calls by **16%** and wall-clock time by **34%**, while RHO's full multi-file repository evolution achieved the inverse (**27%** fewer tool calls and **20%** less wall-clock time at a near-doubling of held-out success). When given full repository access, the agent optimizes prompts and control methods where appropriate. For example, in the GEPA-style single-file setting, retries for tool failures must be described in prompts, forcing the LLM in the control loop to handle recovery. In the multi-file RHO setting, the coding agent autonomously determines that the best approach is to move retries into the low-level APIs and write hardened methods.

Table 3: The structural prior at the seed and LLM capabilities determine the terminal frontier on CaP-Bench Robosuite S4. The multi-file repository prior dominates; rigid program-shape scaffolds collapse. The Codex single file row is reported as cumulative across a 100-generation parent and a 150-generation warm-restart on the remaining hard tasks after saturating the easy ones.

Structural prior	gens	input (M)	cached (M)	output (M)	cost (USD)	Robosuite S4
Multi-file repository (Codex)	200	625.28	88.85	3.14	$2,821	**70.00%**
Single file (Codex)	250	523.14	229.44	2.71	$1,665	63.86%
Behavior Tree (Claude Code Sonnet 4.6)	150	—	3.33	3.98	$124	33.86%
Finite State Machine (Claude Code & Qwen3.6-27B)	200	244.91	—	3.16	$0	18.14%

4.4 RQ4: Do RHO candidate solutions generalize?

Beyond CaP-Bench's primitive interface, RHO transfers to Robotec's RAI ROS 2 manipulation stack (O3DE) [17] and improves accuracy while reducing inference cost. The same algorithm reaches **0.889** on easy (from **0.683**) and **0.443** on hard (from **0.235**), both with $p < 0.001$ (n=21, 3 runs each). The core RHO loop is unchanged; the RAI run changes the candidate artifact and adds the fixed tool-name contract. A single-file ablation (which strips repository-level mutation and recovers a GEPA-style coding-agent loop) loses **0.10** on easy and **0.07** on hard while inflating inference cost on hard (**+16%** tool calls, **+34%** wall-clock); the full algorithm trims tool calls by **27%** and wall-clock by **20%** on the same hard subset (App. A.3.5). Both selected candidates emerge within a **10-generation training budget**: the ablation at generation 2, the full algorithm at generation 8.

5 Discussion

A qualitative analysis of the trajectories and final candidates from our CaP-Bench experiments reveals what the surviving repositories have in common, and where they differ. Given enough computational budget, all approaches begin to produce candidates with desirable properties. We find that all approaches in Table 3 produce candidates that have the same core components: prompt routing, text-prompted segmentation, image-to-world 3D conversion, grasp planning with fallback, and re-observe-then-verify logic. The differences lie in how these components are combined, how pre-gating and planning are handled, and how robust they are to failures. The exploration afforded by repository-level searches is what enables these differences to emerge. For example, the strongest solver alone exhibits the following behaviors: held-object offset compensation (measuring the space between the gripper and object *after successful grasping*), quantile-robust geometry (using quantiles vs means to overcome segmentation mask bleeding), a score-gated perception cascade (falling back from text-prompted segmentation to Molmo point prompts, across multiple cameras, only when the detector score drops below a threshold it discovers), physical-plausibility checks on perception (rejecting implausible object extents and re-ranking detections by whether the grasp point lies inside the object box), tool-center-point offset correction along the approach axis, inverse kinematics (IK)-feasibility pre-gating (accept a grasp *iff* the grasp pose and lifted pose solve IK), dual-perception cross-validation that distrusts its own re-observation (cross-checking text-prompt detections against point-prompt detections and flagging stale perception), and verification that triggers corrective retries with new plans (rather than retrying the same failed plan and actions). As RHO produces neurosymbolic code, it can discover and encode these advanced heuristics and patterns in a way that is both interpretable and reusable. Recovering and checking comparable logic inside an end-to-end neural policy like a VLA is a separate problem in interpretability and formal verification [91, 92].

On the RAI hard O3DE split, repository-level mutation changes *where* recovery logic lives, not only how much of it exists. The single-file evolution improves accuracy by asking the LLM to re-perceive and persist between moves, which raises tool calls and wall time. However, the multi-file candidate instead moves part of that recovery behind a ROS 2 tool API. In particular, the evolved perception wrapper retries detector query variants and *caches* verified centroids before the LLM sees the failure, making a single tool call more consistent, discriminating, and faster.

A final lesson concerns *where* environment interaction is paid. Table 4 counts how often each backend's mutator probed the live environment during search, by running candidates, invoking primitives, querying the visual-difference module (VDM), and re-observing the scene. All four channels rank-order with deployed success, and only the largest frontier mutators (Codex `gpt-5.5`, Claude Code `Opus 4.7`) query visual feedback at volume. An LLM cannot write robust interactive robot code zero-shot: CaP-Agent0 needs its multi-turn visual-difference loop and collapses from 68.29% to 24% without it [4], and the strongest HELIX repositories are those whose search engaged that feedback most. RHO relocates this interaction rather than removing it. The experience CaP-Agent0 gathers turn-by-turn at deployment and repeats across trials, RHO gathers *once* during evolutionary search and compiles into the shipped repository for CaP-Bench, which deploys with no corrective code edits from an LLM. Appendix A.2.6 offers a deeper analysis of Table 4.

In both settings these interpretable neurosymbolic mechanisms are discovered rather than designed. Searching multi-file repositories instead of single prompts or functions is what lets RHO produce robust policies.

6 Conclusion

RHO is an evolutionary reflective optimizer for proposing and finding interpretable neurosymbolic multi-file robot-policies (*Repositories-as-Policies*). In this paper, our RHO instantiation combines HELIX, REaaS, and evaluator isolation: HELIX supplies repository-level coding-agent mutation, REaaS exposes robot-environment rollouts during search, and isolation separates mutation from scoring to prevent cheating without performance degradation. RHO sets the state-of-the-art on CaP-Bench under single-turn deployment with no LLM code-generation calls in the control loop: **70.0%** on Robosuite (vs CaP-Agent0's **68.29%** multi-turn record at up to **66** LLM calls per trial) and **45.0%** on LIBERO-PRO perturbations (vs **18.17%** for the prior agent, **0.0%** for OpenVLA, and **12.83%** for $\pi_{0.5}$). The same RHO loop transfers to Robotec's RAI ROS 2 stack, boosting task success while trimming tool calls by **27%** and wall-clock time by **20%** on the hard subset, while a single-file ablation of repository-level mutation inflates both metrics (**+16%** and **+34%**). An interesting future direction is to extend RHO from its current scalar reward and per-instance coverage frontier to vector-valued objectives with secondary scores such as repository size, execution latency, and overall tool or service count used at deployment. Another immediate direction is to deploy RHO's RAI solution to production environments, as it is already built on ROS 2.

7 Limitations

RHO inherits the spatial-reasoning limits of the LLM that drives its search: current models misjudge quantitative depth, vertical clearance, and multi-object spatial composition under modest perturbations [93, 94, 95], so tasks that turn on fine geometric reasoning are the natural ceiling for a

Table 4: Training-time robotics tool use across backends and configurations. Each cell counts REaaS calls to one endpoint issued by the mutator during the run: `reset` (per-trial setup), `run` (candidate execution), `call` (perception/action primitive), `vdm` (visual-difference queries), and `observe` (scene re-observation). Section A.2.6 analyzes how these channels, especially `vdm`, track deployed success; token and cost totals appear in Tables 2 and 3.

Run (S4, Robosuite)	reset	run	call	vdm	observe	total
Multi-file repository (Codex) (70.0%)	698	576	293	166	43	1,776
Single file (Codex) (63.9%)	612	486	247	86	31	1,462
Claude Code Opus 4.7 (61.9%)	518	405	182	84	31	1,220
Qwen3.6-27B (via Claude Code), free-form (35.0%)	100	61	32	8	8	209
Behavior Tree (Claude Code Sonnet 4.6) (33.9%)	14	11	3	0	0	28
Qwen3.6-27B (via Claude Code), FSM (18.1%)	57	5	0	0	0	62

code-writing agent, which stronger spatially grounded models would relax. All of our benchmarks are simulated, and the RHO instances studied here emit interpretable neurosymbolic policies: symbolic primitive-calling code over learned perception and grasping services plus motion primitives. RHO could optimize repositories that train neural network components, but our reported systems do not train an end-to-end neural policy on the simulator. We therefore expect the shipped policies to transfer to real robots once their perception primitives are re-grounded and the constants they fit in simulation are re-tuned [1, 2, 96]. Granting mutators shell access also exposes the experimental infrastructure itself, not only the reward signal: in one preliminary run an agent told to stop hardcoding per-trial solutions instead terminated the operator's SSH session and fork-bombed the host. We treat natural-language correction as insufficient containment for optimization-driven agents, which motivates the hard isolation boundary of Section 3.2.

Acknowledgments

This work was supported by gifts from Accenture, Algorithmic SuperIntelligence Labs, Amazon, AMD, Anyscale, Broadcom, cmpnd, Google, IBM, Intel, Intesa Sanpaolo, Lambda, Lightspeed, NVIDIA, Samsung SDS, and SAP.

References

[1] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023. doi:10.48550/arXiv.2209.07753. URL https://arxiv.org/abs/2209.07753.

[2] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. ProgPrompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. doi:10.48550/arXiv.2209.11302. URL https://arxiv.org/abs/2209.11302.

[3] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023. doi:10.48550/arXiv.2307.05973. URL https://arxiv.org/abs/2307.05973.

[4] M. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, F.-F. Li, G. Shi, J. Wu, S. Sastry, Y. Zhu, K. Goldberg, and L. Fan. CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation. *arXiv preprint arXiv:2603.22435*, 2026. doi:10.48550/arXiv.2603.22435. URL https://arxiv.org/abs/2603.22435.

[5] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025. URL https://arxiv.org/abs/2510.03827.

[6] L. Chen, J. Q. Davis, B. Hanin, P. Bailis, I. Stoica, M. Zaharia, and J. Zou. Are more llm calls all you need? towards scaling laws of compound inference systems. *arXiv preprint arXiv:2403.02419*, 2024. URL https://arxiv.org/abs/2403.02419.

[7] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024. doi:10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

[8] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair,

K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *Conference on Robot Learning (CoRL), Oral*, 2025. doi:10.48550/arXiv.2504.16054. URL https://arxiv.org/abs/2504.16054.

[9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.03310.

[10] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models. In *ICLR 2026*, 2026. doi:10.48550/arXiv.2510.13626. URL https://arxiv.org/abs/2510.13626. Title corrected to canonical arXiv title (was "Stress-Testing ... under Perturbation"). Venue "ICLR 2026" is as-claimed in refs.bib; not DBLP-confirmed.

[11] G. Wang, C. Zhang, Q. Liu, J. Zhang, J. Cai, J. Liu, and X. Liu. LIBERO-X: Robustness litmus for vision-language-action models. *arXiv preprint arXiv:2602.06556*, 2026. doi:10.48550/arXiv.2602.06556. URL https://arxiv.org/abs/2602.06556.

[12] C. Kim, M. Kim, M. Kang, H. Kim, and D. Jung. LIBERO-Para: A diagnostic benchmark and metrics for paraphrase robustness in VLA models. *arXiv preprint arXiv:2603.28301*, Mar. 2026. doi:10.48550/arXiv.2603.28301. URL https://arxiv.org/abs/2603.28301.

[13] K. Elmaaroufi, M. Zaharia, and S. A. Seshia. LIBERO-Infinity: A Scenic-3-Based Open-Ended Evaluation Harness for VLAs. https://github.com/KE7/libero-infinity, Mar. 2026. v0.1.0 released Mar 16 2026; 9 composable perturbation axes. Lead author verified via repo; full co-author list not independently checkable (no arXiv/DBLP record).

[14] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR), Oral*, 2026. doi:10.48550/arXiv.2507.19457. URL https://arxiv.org/abs/2507.19457. arXiv:2507.19457. Underlying reflective optimizer that Optimize-Anything generalizes.

[15] L. A. Agrawal, D. Lee, W. Ma, K. Elmaaroufi, S. Tan, S. A. Seshia, K. Sen, D. Klein, I. Stoica, J. E. Gonzalez, O. Khattab, A. G. Dimakis, and M. Zaharia. optimize_anything: A universal api for optimizing any text parameter. In *Proceedings of the ACM Conference on AI and Agentic Systems (CAIS 2026), Demo Track*, San Jose, California, USA, May 2026. ACM. URL https://www.caisconf.org/program/2026/demos/optimize-anything/. Demo paper. May 27–29, 2026.

[16] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. URL https://arxiv.org/abs/2009.12293.

[17] K. Rachwał, M. Majek, B. Boczek, K. Dąbrowski, P. Liberadzki, A. Dąbrowski, and M. Ganzha. RAI: Flexible agent framework for embodied AI, 2025. URL https://arxiv.org/abs/2505.07532.

[18] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023. doi:10.48550/arXiv.2212.06817. URL https://arxiv.org/abs/2212.06817.

[19] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. doi:10.48550/arXiv.2307.15818. URL https://arxiv.org/abs/2307.15818.

[20] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024. URL https://arxiv.org/abs/2405.12213.

[21] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. doi:10.48550/arXiv.2410.24164. URL https://arxiv.org/abs/2410.24164.

[22] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. RDT-1B: A diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.07864.

[23] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025. doi:10.48550/arXiv.2501.15830. URL https://arxiv.org/abs/2501.15830.

[24] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna. MolmoAct: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025. doi:10.48550/arXiv.2508.07917. URL https://arxiv.org/abs/2508.07917.

[25] NVIDIA, J. Bjorck, F. C. neda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. doi:10.48550/arXiv.2503.14734. URL https://arxiv.org/abs/2503.14734.

[26] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Robotics: Science and Systems (RSS)*, 2025. doi:10.48550/arXiv.2502.19645. URL https://arxiv.org/abs/2502.19645.

[27] H. Zhen, X. Qiu, P. Chen, J. Yang, X. Yan, Y. Du, Y. Hong, and C. Gan. 3D-VLA: A 3D vision-language-action generative world model. In *International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 61229–61245, 2024. URL https://arxiv.org/abs/2403.09631.

[28] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025. doi:10.48550/arXiv.2506.21539. URL https://arxiv.org/abs/2506.21539.

[29] H. Chen, J. Guo, B. Wang, T. Zhang, X. Huang, B. Zheng, Y. Hou, C. Tie, J. Deng, and L. Shao. Goal-VLA: Image-Generative VLMs as Object-Centric World Models Empowering Zero-shot Robot Manipulation. *arXiv preprint arXiv:2506.23919*, 2026. doi:10.48550/arXiv.2506.23919. URL https://arxiv.org/abs/2506.23919.

[30] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. In *International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2310.16828. URL https://arxiv.org/abs/2310.16828.

[31] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering Diverse Domains through World Models. *Nature*, 640:647–653, 2025. doi:10.1038/s41586-025-08744-2. URL https://arxiv.org/abs/2301.04104. Published in Nature 640:647–653 (2025) under the title "Mastering diverse control tasks through world models"; arXiv preprint title retained here as cited.

[32] P. Wu, A. Escontrela, D. Hafner, K. Goldberg, and P. Abbeel. DayDreamer: World models for physical robot learning. In *Conference on Robot Learning (CoRL)*, 2022. doi:10.48550/arXiv.2206.14176. URL https://arxiv.org/abs/2206.14176.

[33] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan. RoboDreamer: Learning compositional world models for robot imagination. In *International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 61885–61896, 2024. URL https://arxiv.org/abs/2404.12377.

[34] Z. Mei, T. Yin, M. Baker, O. Shorinwa, and A. Majumdar. World models that know when they don't know – controllable video generation with calibrated uncertainty, 2025. URL https://arxiv.org/abs/2512.05927. arXiv records year 2026 for the revised version; eprint 2512.05927 is the Dec-2025 v1.

[35] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated Task and Motion Planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021. doi:10.1146/annurev-control-091420-084139.

[36] S. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor. ChatGPT for robotics: Design principles and model abilities. *arXiv preprint arXiv:2306.17582*, 2023. doi:10.48550/arXiv.2306.17582. URL https://arxiv.org/abs/2306.17582.

[37] S. Huang, Z. Jiang, H. Dong, Y. Qiao, P. Gao, and H. Li. Instruct2Act: Mapping multi-modality instructions to robotic actions with large language model. *arXiv preprint arXiv:2305.11176*, 2023. doi:10.48550/arXiv.2305.11176. URL https://arxiv.org/abs/2305.11176.

[38] Z. Hu, F. Lucchetti, C. Schlesinger, Y. Saxena, A. Freeman, S. Modak, A. Guha, and J. Biswas. Deploying and Evaluating LLMs to Program Service Mobile Robots. *IEEE Robotics and Automation Letters (RA-L)*, 2024. doi:10.1109/LRA.2024.3360020. URL https://arxiv.org/abs/2311.11183.

[39] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022. doi:10.48550/arXiv.2204.01691. URL https://arxiv.org/abs/2204.01691.

[40] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter. Inner monologue: Embodied reasoning through planning with language models. In *Conference*

on Robot Learning (CoRL), 2022. doi:10.48550/arXiv.2207.05608. URL https://arxiv.org/abs/2207.05608.

[41] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. In *Conference on Robot Learning (CoRL)*, 2023. doi:10.48550/arXiv.2307.06135. URL https://arxiv.org/abs/2307.06135.

[42] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu, Q. Vuong, T. Zhang, T.-W. E. Lee, K.-H. Lee, P. Xu, S. Kirmani, Y. Zhu, A. Zeng, K. Hausman, N. Heess, C. Finn, S. Levine, and B. Ichter. PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.07872.

[43] F. Liu, K. Fang, P. Abbeel, and S. Levine. MOKA: Open-world robotic manipulation through mark-based visual prompting. In *Robotics: Science and Systems (RSS)*, 2024. URL https://arxiv.org/abs/2403.03174.

[44] B. Y. Tsui, A. Y. Fang, and T. J. Hwu. Demonstration-free robotic control via LLM agents, 2026. URL https://arxiv.org/abs/2601.20334.

[45] V. G. dos Santos, I. Khadraoui, I. Farhat, H. Yous, S. Teffahi, and H. Hacid. ALRM: Agentic LLM for robotic manipulation, 2026. URL https://arxiv.org/abs/2601.19510.

[46] V. Kumar. Act-observe-rewrite: Multimodal coding agents as in-context policy learners for robot manipulation. *arXiv preprint arXiv:2603.04466*, 2026. doi:10.48550/arXiv.2603.04466. URL https://arxiv.org/abs/2603.04466.

[47] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.12931.

[48] Y. J. Ma, W. Liang, H.-J. Wang, S. Wang, Y. Zhu, L. Fan, O. Bastani, and D. Jayaraman. DrEureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024. URL https://arxiv.org/abs/2406.01967.

[49] W. Liang, S. Wang, H.-J. Wang, O. Bastani, D. Jayaraman, and Y. J. Ma. Eurekaverse: Environment curriculum generation via large language models. In *Conference on Robot Learning (CoRL)*, 2024. doi:10.48550/arXiv.2411.01775. URL https://arxiv.org/abs/2411.01775.

[50] K. Ryu, Q. Liao, Z. Li, P. Delgosha, K. Sreenath, and N. Mehr. CurricuLLM: Automatic task curricula design for learning complex robot skills using large language models. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2025. doi:10.48550/arXiv.2409.18382. URL https://arxiv.org/abs/2409.18382.

[51] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. Gonzalez Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, B. Ichter, T. Xiao, P. Xu, A. Zeng, T. Zhang, N. Heess, D. Sadigh, J. Tan, Y. Tassa, and F. Xia. Language to rewards for robotic skill synthesis. In *Proc. Conference on Robot Learning (CoRL)*, pages 374–404, 2023. URL https://arxiv.org/abs/2306.08647.

[52] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu. Text2Reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2309.11489. URL https://arxiv.org/abs/2309.11489.

[53] H. Field, M. Yang, Y. Lin, E. Psomopoulou, D. Barton, and N. F. Lepora. Text2Touch: Tactile In-Hand Manipulation with LLM-Designed Reward Functions. In *Proc. Conference on Robot Learning (CoRL)*, 2025. doi:10.48550/arXiv.2509.07445. URL https://arxiv.org/abs/2509.07445.

[54] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires. REvolve: Reward Evolution with Large Language Models using Human Feedback. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. doi:10.48550/arXiv.2406.01309. URL https://arxiv.org/abs/2406.01309.

[55] C. Huang, Y. Chang, J. Lin, J. Liang, R. Zeng, and J. Li. Efficient language-instructed skill acquisition via reward-policy co-evolution. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 39, pages 14576–14584, 2025. doi:10.48550/arXiv.2412.13492. URL https://arxiv.org/abs/2412.13492.

[56] C. Huang, J. Liang, Y. Chang, J. Xu, and J. Li. Automated hybrid reward scheduling via large language models for robotic skill learning. *arXiv preprint arXiv:2505.02483*, 2025. doi:10.48550/arXiv.2505.02483. URL https://arxiv.org/abs/2505.02483.

[57] J. Rocamonde, V. Montesinos, E. Nava, E. Perez, and D. Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *Proc. International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2310.12921. URL https://arxiv.org/abs/2310.12921.

[58] S. Patel, X. Yin, W. Huang, S. Garg, H. Nayyeri, L. Fei-Fei, S. Lazebnik, and Y. Li. A real-to-sim-to-real approach to robotic manipulation with VLM-generated iterative keypoint rewards. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2025. URL https://arxiv.org/abs/2502.08643.

[59] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan. RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation. In *International Conference on Machine Learning (ICML)*, 2024. doi:10.48550/arXiv.2311.01455. URL https://arxiv.org/abs/2311.01455.

[60] L. Wang, Y. Ling, Z. Yuan, M. Shridhar, C. Bao, Y. Qin, B. Wang, H. Xu, and X. Wang. GenSim: Generating robotic simulation tasks via large language models. In *International Conference on Learning Representations (ICLR)*, 2024. doi:10.48550/arXiv.2310.01361. URL https://arxiv.org/abs/2310.01361.

[61] P. Hua, M. Liu, A. Macaluso, Y. Lin, W. Zhang, H. Xu, and L. Wang. GenSim2: Scaling robot data generation with multi-modal and reasoning LLMs. In *Proc. Conference on Robot Learning (CoRL)*, pages 5030–5066, 2024. URL https://arxiv.org/abs/2410.03645.

[62] Y. Yang, F.-Y. Sun, L. Weihs, E. VanderBilt, A. Herrasti, W. Han, J. Wu, N. Haber, R. Krishna, L. Liu, C. Callison-Burch, M. Yatskar, A. Kembhavi, and C. Clark. Holodeck: Language guided generation of 3D embodied AI environments. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. doi:10.48550/arXiv.2312.09067. URL https://arxiv.org/abs/2312.09067.

[63] A. Zala, J. Cho, H. Lin, J. Yoon, and M. Bansal. EnvGen: Generating and adapting environments via LLMs for training embodied agents. In *Proc. Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2403.12014.

[64] N. Gao, Y. Chen, S. Yang, X. Chen, Y. Tian, H. Li, H. Huang, H. Wang, T. Wang, and J. Pang. GenManip: LLM-driven Simulation for Generalizable Instruction-Following Manipulation. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. doi:10.48550/arXiv.2506.10966. URL https://arxiv.org/abs/2506.10966.

[65] C. Yao, X. Liu, C. Li, and M. Savvides. Reward evolution with graph-of-thoughts: A bi-level language model framework for reinforcement learning. *arXiv preprint arXiv:2509.16136*, 2025. URL https://arxiv.org/abs/2509.16136.

[66] B. Wu. MAESTRO: Multi-agent environment shaping through task and reward optimization. *arXiv preprint arXiv:2511.19253*, 2025. doi:10.48550/arXiv.2511.19253. URL https://arxiv.org/abs/2511.19253.

[67] Z. Wu, J. Lu, Y. Chen, Y. Liu, Y. Zhuang, and L. Hu. STRIDE: Automating reward design, deep reinforcement learning training and feedback optimization in humanoid robotics locomotion. *arXiv preprint arXiv:2502.04692*, 2025. doi:10.48550/arXiv.2502.04692. URL https://arxiv.org/abs/2502.04692.

[68] Z. Zhao, C. Hua, F. Berto, Z. Ma, K. Lee, J. Li, and J. Park. EvoNav: Evolutionary reward function design for robot navigation with large language models. *arXiv preprint arXiv:2605.11859*, 2026. URL https://arxiv.org/abs/2605.11859.

[69] J. Fang, Y. Sun, C. Ma, Q. Lu, and L. Yao. RoboMoRe: LLM-based Robot Co-design via Joint Optimization of Morphology and Reward. *arXiv preprint arXiv:2506.00276*, 2025. doi:10.48550/arXiv.2506.00276. URL https://arxiv.org/abs/2506.00276.

[70] T. Xu, H. Wen, and M. Li. Adapting the interface, not the model: Runtime harness adaptation for deterministic llm agents. *arXiv preprint arXiv:2605.22166*, 2026. doi:10.48550/arXiv.2605.22166. URL https://arxiv.org/abs/2605.22166.

[71] Y. Zhang, J. Wang, Y. Ge, W. Xu, J. Hamm, and C. K. Reddy. Stop comparing llm agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026. URL https://arxiv.org/abs/2605.23950.

[72] L. Pan, L. Zou, S. Guo, J. Ni, and H.-T. Zheng. Natural-language agent harnesses, 2026. URL https://arxiv.org/abs/2603.25723.

[73] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature*, 645:633–638, 2025. doi:10.1038/s41586-025-09422-z. URL https://arxiv.org/abs/2501.12948.

[74] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026. doi:10.48550/arXiv.2603.28052. URL https://arxiv.org/abs/2603.28052.

[75] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, J. Jitsev, D. Lu, O. Menis Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, J. Hu, C. M. Rytting, R. Marten, Y. Wang, A. Dimakis, A. Konwinski, and L. Schmidt. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026. doi:10.48550/arXiv.2601.11868. URL https://arxiv.org/abs/2601.11868.

[76] A. L. Zhang, T. Kraska, and O. Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2025. URL https://arxiv.org/abs/2512.24601.

[77] D. Wang. Think, but don't overthink: Reproducing recursive language models. *arXiv preprint arXiv:2603.02615*, 2026. doi:10.48550/arXiv.2603.02615. URL https://arxiv.org/abs/2603.02615.

[78] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi:10.1038/s41586-023-06924-6. FunSearch; published online 14 Dec 2023.

[79] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

[80] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi:10.48550/arXiv.2303.11366. URL https://arxiv.org/abs/2303.11366.

[81] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Transactions on Machine Learning Research (TMLR)*, 2024. doi:10.48550/arXiv.2305.16291. URL https://arxiv.org/abs/2305.16291. Originally arXiv:2305.16291, 2023.

[82] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025. URL https://arxiv.org/abs/2505.22954.

[83] J. Nam, J. Yoon, J. Chen, J. Shin, S. Ö. Arık, and T. Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement. *arXiv preprint arXiv:2506.15692*, 2025. doi:10.48550/arXiv.2506.15692. URL https://arxiv.org/abs/2506.15692.

[84] Y. Wang, S. Liu, J. Fang, and Z. Meng. EvoAgentX: An automated framework for evolving agentic workflows. *arXiv preprint arXiv:2507.03616*, 2025. doi:10.48550/arXiv.2507.03616. URL https://arxiv.org/abs/2507.03616.

[85] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. doi:10.48550/arXiv.2408.03314. URL https://arxiv.org/abs/2408.03314. Published at ICLR 2025 (proceedings title appends "for Reasoning").

[86] OpenAI. OpenAI o1 system card, 2024. URL https://arxiv.org/abs/2412.16720. System card dated 2024-12-05.

[87] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.10601.

[88] Y. Atinafu and R. Cohen. RewardHackingAgents: Benchmarking evaluation integrity for LLM ML-engineering agents. *arXiv preprint arXiv:2603.11337*, Mar. 2026. doi:10.48550/arXiv.2603.11337. URL https://arxiv.org/abs/2603.11337. Evaluator-tampering attempts occur in about 50% of natural-agent episodes and are eliminated by evaluator locking with 25–31% median runtime overhead.

[89] OpenAI. Codex cli: Lightweight coding agent that runs in your terminal. https://github.com/openai/codex, Apr. 2025. URL https://github.com/openai/codex. Accessed 2026-05-28.

[90] Anthropic. Claude 3.7 sonnet and claude code. https://www.anthropic.com/news/claude-3-7-sonnet, Feb. 2025. URL https://www.anthropic.com/news/claude-3-7-sonnet. Accessed 2026-05-28.

[91] C. Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019. doi:10.1038/s42256-019-0048-x.

[92] S. A. Seshia, D. Sadigh, and S. S. Sastry. Toward verified artificial intelligence. *Communications of the ACM*, 65(7):46–55, 2022. doi:10.1145/3503914.

[93] R. Xu, W. Wang, H. Tang, X. Chen, X. Wang, F.-J. Chu, D. Lin, M. Feiszli, and K. J. Liang. Multi-SpatialMLLM: Multi-frame spatial understanding with multi-modal large language models. *arXiv preprint arXiv:2505.17015*, 2025. doi:10.48550/arXiv.2505.17015. URL https://arxiv.org/abs/2505.17015.

[94] M. Gholami, A. Rezaei, Z. Weimin, S. Mao, S. Zhou, Y. Zhang, and M. Akbari. Spatial Reasoning with Vision-Language Models in Ego-Centric Multi-View Scenes. *arXiv preprint arXiv:2509.06266*, 2025. doi:10.48550/arXiv.2509.06266. URL https://arxiv.org/abs/2509.06266.

[95] P. Chen, Y. Lou, S. Cao, J. Guo, L. Fan, Y. Wu, L. Yang, L. Ma, and J. Ye. SD-VLM: Spatial measuring and understanding with depth-encoded vision-language models. *arXiv preprint arXiv:2509.17664*, 2025. doi:10.48550/arXiv.2509.17664. URL https://arxiv.org/abs/2509.17664. Accepted by NeurIPS 2025.

[96] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser. TidyBot: Personalized robot assistance with large language models. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023. doi:10.48550/arXiv.2305.05658. URL https://arxiv.org/abs/2305.05658.

[97] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, J. Lei, T. Ma, B. Guo, A. Kalla, M. Marks, J. Greer, M. Wang, P. Sun, R. Rädle, T. Afouras, E. Mavroudi, K. Xu, T.-H. Wu, Y. Zhou, L. Momeni, R. Hazra, S. Ding, S. Vaze, F. Porcher, F. Li, S. Li, A. Kamath, H. K. Cheng, P. Dollár, N. Ravi, K. Saenko, P. Zhang, and C. Feichtenhofer. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. doi:10.48550/arXiv.2511.16719. URL https://arxiv.org/abs/2511.16719.

[98] M. Deitke, C. Clark, S. Lee, R. Tripathi, Y. Yang, J. S. Park, M. Salehi, N. Muennighoff, K. Lo, L. Soldaini, J. Lu, T. Anderson, E. Bransom, K. Ehsani, H. Ngo, Y. Chen, A. Patel, M. Yatskar,

C. Callison-Burch, A. Head, R. Hendrix, F. Bastani, E. VanderBilt, N. Lambert, Y. Chou, A. Chheda, J. Sparks, S. Skjonsberg, M. Schmitz, A. Sarnat, B. Bischoff, P. Walsh, C. Newell, P. Wolters, T. Gupta, K.-H. Zeng, J. Borchardt, D. Groeneveld, C. Nam, S. Lebrecht, C. Wittlif, C. Schoenick, O. Michel, R. Krishna, L. Weihs, N. A. Smith, H. Hajishirzi, R. Girshick, A. Farhadi, and A. Kembhavi. Molmo and PixMo: Open weights and open data for state-of-the-art vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. URL https://arxiv.org/abs/2409.17146.

[99] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer. SAM 2: Segment anything in images and videos. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. doi:10.48550/arXiv.2408.00714. URL https://arxiv.org/abs/2408.00714.

A Appendix

A.1 Methodology Appendix

This appendix collects the material deferred from Sec. 3: regime definitions, notation, API surface, loop mechanics, sandbox evidence, run provenance, and RAI portability. The canonical RHO run selects candidate `g134-s134`; its mutable deployment artifact is the `solver/` Python package, evaluated by `evaluate.py` under the `API_REFERENCE.md` contract and the reported `helix.toml` settings: `backend="codex"`, `model="gpt-5.5"`, `effort="xhigh"`, `max_turns=150`.

Regime definitions and primitive registry. M4 is the multi-turn training regime: a coding-agent session may consume execution feedback, visual-difference summaries, judge diagnostics, and stdout/stderr when proposing a source edit. S4 is the single-turn deployment regime, in which the selected source tree is frozen and invoked through `solver.main.run(ctx)`. Validation produces the internal selection numbers; VDM informs mutation but is withheld at the S4 deployment surface and never enters acceptance. The external S4 replay is the deployment evaluation.

Notation and selector inputs. Let $\mathfrak{L}_{\text{py}}$ be the universe of admissible Python solver repositories and $\mathcal{L} \in \mathfrak{L}_{\text{py}}$ a candidate library. The seven CaP-X task families are $\mathcal{T} = \{\text{cube_lifting}, \text{cube_stack}, \text{cube_restack}, \text{nut_assembly}, \text{spill_wipe}, \text{two_arm_lift}, \text{two_arm_handover}\}$, and a trial is $\tau = (t, k)$ with $t \in \mathcal{T}$. The M4 train and validation splits each contain $7 \times 10{=}70$ trials; the CaP-Bench headline uses $7 \times 100{=}700$ trials, not 700 distinct tasks. The M4 train and validation splits draw the same per-task trial IDs $\{1, \ldots, 10\}$, so RHO never sees the remaining 90 trial IDs per task; the S4 headline scores the frozen solver on the full 7×100 set, of which $630/700$ trials are held out. The evaluator returns trajectory $E(\mathcal{L}, \tau)$, scalar shaped reward $r_\tau(\mathcal{L}) = R(E(\mathcal{L}, \tau)) \in [0, 1]$, task-success threshold $\theta_\star = 1.0$, and ASI s_τ containing task completion, optional M4-only visual-difference summary s_τ^{vdm}, judge axes $j_\tau \in [0, 1]^5$, and an integrity flag. The coding agent may extend s_τ with trajectory-derived ASI it authors itself.

Deployed artifact inventory and API accounting. The selected package `solver/` is a multi-file Python package (Figure 5) containing `main.py` (task router and `run(ctx)` entry point), `common.py` and `robotics.py` (shared helpers for perception, grasp planning, motion, and placement), and a `tasks/` subpackage with per-task routines. The shared helpers include `_observe`, `_segment_object`, `_grasp_from_mask`, `_approach_grasp_lift`, `_place_on_target`, and `_normalize_prompt`. `run(ctx)` parses the language instruction, takes one observation through `_observe`, dispatches to the relevant task routine, and returns a status dictionary. The implementation has no solver-owned external services, communicating only through stdout and `ctx`.

The import audit finds only the stdlib and `numpy`; `openai`, `anthropic`, `google.genai`, `litellm`, `requests`, `httpx`, `urllib`, `socket`, `subprocess`, `capx`, `sidecar`, and `tools.capx_tool` are absent. The source audit also finds no solver-owned visual-debate rollout, runtime skill-retrieval index, embedding store, parallel LLM voting layer, or LLM HTTP/CLI clients.

The S4 runtime API supplies `get_observation`, SAM 3 text and point segmentation [97], Molmo point prompting [98], Contact-GraspNet grasps, oriented bounding boxes, IK, joint motion, gripper commands, and bimanual variants. `get_observation()` returns RGB, depth, camera intrinsics, and camera-to-world pose under `obs["robot0_robotview"]`; missing fields produce `{"status": "no_observation"}`. Contact-GraspNet candidate poses and scores are returned in the camera frame and composed with the camera pose before IK. Quaternions are WXYZ throughout. Training-distilled helpers for object features, grasp choice, pick/lift, placement, WXYZ conversion, OBB construction, and finite-state task branches live as Python functions in the package; there is no runtime retrieval surface.

Reproducibility manifest. The HELIX loop uses GEPA-style strict-improvement acceptance [14], `frontier_type="instance"`, `minibatch_size=2`, a stratified sampler,

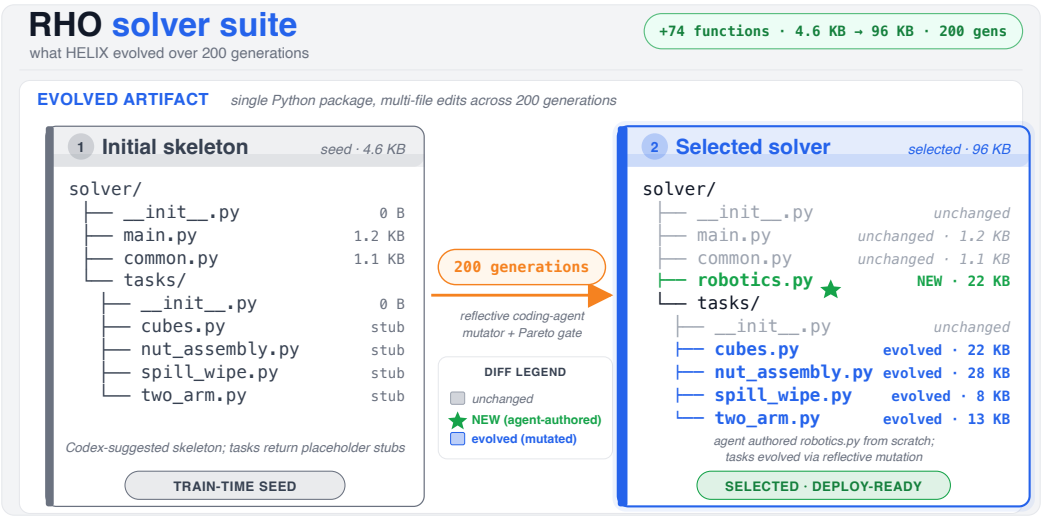


Figure 5: Deployed solver-suite interface: `run(ctx)`, task routines, and shared helpers; all benchmark interaction goes through `ctx`.

`perfect_score_threshold=1.0`, and disabled merge/crossover in the reported run. The final state records `generation=200`, `mutation_counter=200`, `merge_counter=0`, 87 retained frontier candidates, and 70 active validation keys.

Evaluator contract and reflection. The evaluator reads HELIX's `helix_batch.json`, preserves its order, resolves each id to a native CaP-X task/trial, runs `solver.main.run(ctx)`, and emits one positional line:

```
HELIX_RESULT=[[score_0, side_info_0], [score_1, side_info_1], ...]
```

HELIX zips this payload with the batch ids. The scalar score is the environment-provided shaped final reward. Judge feedback, VDM summaries, stdout/stderr, and `feedback/last_run/` files feed future mutation prompts as diagnostic side information; they are not inputs to the frozen solver at deployment time.

The mutation operator is a bounded autonomous coding-agent session over a copied parent worktree (Figure 6). The reported run uses $b =$ `codex` on `gpt-5.5` with `model_reasoning_effort=xhigh` and K_{turns}=150, invoked as `codex exec -json`; HELIX supports several backends but cross-backend evaluation across $\mathcal{B} = \{\texttt{claude}, \texttt{codex}, \texttt{cursor}, \texttt{gemini}, \texttt{opencode}\}$ is outside the reported run. The mutation session edits only the package surface and terminates when the agent stops issuing tool calls.

The reflection prompt contains task instructions, the parent's current score vector, the machine payload `HELIX_RESULT=[[`r_{τ_1}`,`s_{τ_1}`],...]`, per-trial diagnostics, the M4-only s_τ^{vdm}, judge axes, integrity flags, evaluator notes, and fallback stdout/stderr. Timing and token-cost streams are withheld from the reflection prompt so acceptance cannot reward skipping slow robot operations. The scalar reward remains r_τ; judge axes and VDM summaries are diagnostics only.

A.1.1 Run State, Lineage, and Audit Trail

HELIX treats each coding-agent proposal as a persistent repository snapshot (Figure 2). In the reported root, `.helix/` retains candidate worktrees, accepted evaluation snapshots, `state.json`, `lineage.json`, the evaluator manifest, and the controller log. The re-audit records a single-phase structure: one `state.json`, one `lineage.json`, one `[evolution]` configuration block, and no per-phase directories. `lineage.json` records 166 nodes from `g0-s0` through `g182-s182`;

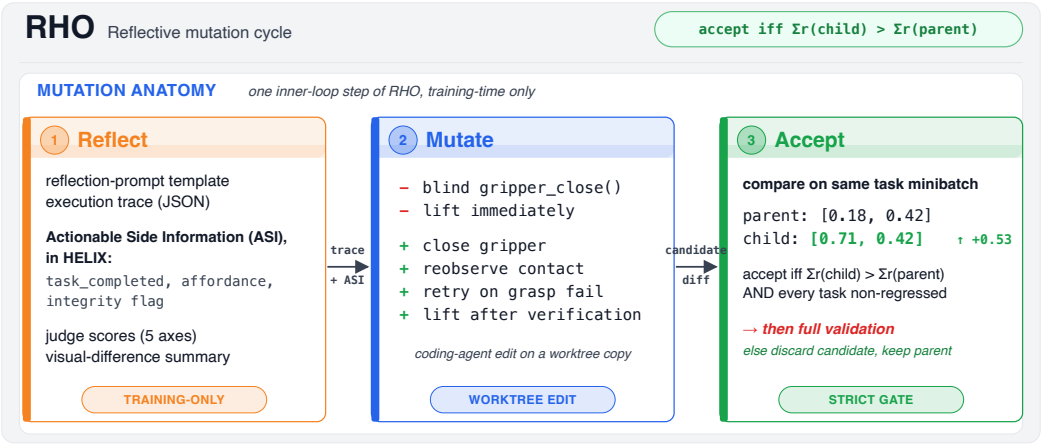


Figure 6: Mutation anatomy: diagnostics and reflection prompt a bounded coding-agent edit; the child repository is accepted only after paired minibatch scoring and validation promotion.

`evaluations/` retains 87 full validation files; `attempts/` retains 77 rejected train-minibatch attempts; and `skips/` retains 28 skip records.

The g1-s1 single-mutation child of the stub seed g0-s0, used as the one-shot Codex baseline in §A.2.8, scores a mean reward of 0.106741 and 0.0% (0/70) task success on the full 70-trial validation set. The selected deployed candidate `g134-s134` scores 0.730982 mean reward and covers 49/70 active instance-frontier keys (70.0%), matching the external S4 headline of 490/700 reported in §4. `g97-s97` appears only as a rejected train-minibatch attempt under `attempts/`: non-canonical historical lineage, not the reported RHO selection. A resumed controller reloads the frontier, verifies the stored `config_hash`, continues mutation and merge counters without ID collisions, and treats worktrees without evaluation snapshots as incomplete proposals.

A.1.2 Configuration, Budgets, and Judge Prompt

The top-level run objective maximizes environment-provided shaped reward for simulated Franka manipulation tasks while preserving `solver.main.run(ctx)` and the documented runtime API. The `[seedless]` block uses `train_path="splits/instance_ids"` and `val_path="splits/instance_ids"`, covering seven task groups with ten native trial labels each. The evaluator command is:

```
python evaluate.py
```

The evaluator uses `score_parser='helix_result'`, protected files `evaluate.py`, `API_REFERENCE.md`, `judge_prompt.md`, `README.md`, `splits`, `helix.toml`, `splits/train.yaml`, and `splits/val.yaml`, and one judge call per candidate. Figure 7 shows the mutator sandboxing enforced around these evaluator-owned files.

Training budgets. The final persisted state reached `generation=200` and `mutation_counter=200`. The retained frontier contains 87 candidates. Lineage over retained candidates comprises 86 mutation operations plus the seed. The selected candidate `g134-s134` has internal best score 0.730982 mean shaped reward and 49/70 trials solved on the 70-instance validation set, and its accepted external S4 replay reports 490/700. The run wall-clock is 79.4 hours from `.helix/config.toml` creation time to the final `.helix/state.json` write, on a dual-socket Intel Xeon Platinum 8570 host with two NVIDIA B200 GPUs. The final state reports `budget.evaluations=6814`, `budget.input_tokens=625276104`, `budget.cached_input_tokens=88851968`, and `budget.output_tokens=3144685`. The sum

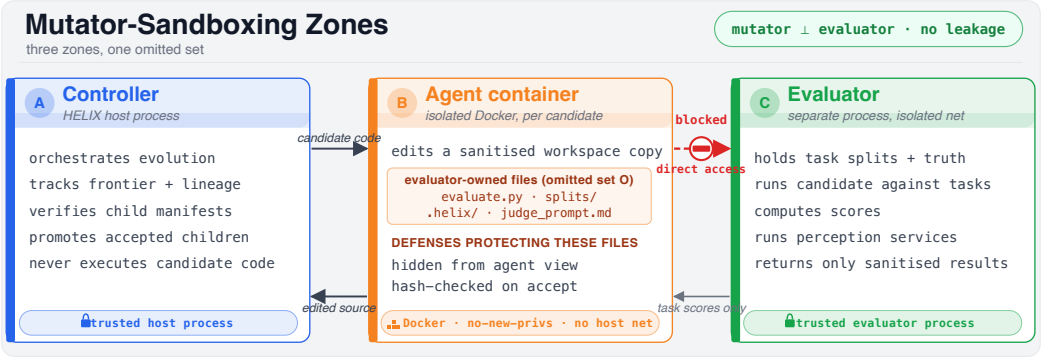


Figure 7: Mutator-sandboxing zones: the agent workspace, evaluator-owned files, candidate execution container, and blocked network edge are separated before acceptance.

over 87 retained backend-result files is 386,757,173 input tokens, 373,486,336 cached input tokens, and 1,757,060 output tokens.

Judge axes. Each accepted candidate is scored by an OpenAI `gpt-5.1` judge on five capability axes plus an integrity flag, returned as a strict-JSON object alongside per-axis evidence citations (code line ranges and trace excerpts). Each axis score is a float in $[0.0, 1.0]$ with anchors at 0.0 (no executed evidence), 0.25 (minimal one-off evidence), 0.5 (clear evidence in one task stage), 0.75 (multiple task-relevant instances), and 1.0 (strong repeated evidence across stages); high task reward alone is explicitly not evidence of any capability. The five axes are *state_uncertainty_management* (re-observation, hypothesis comparison, viewpoint adjustment, delayed irreversible action, belief update), *robustness_contingency_handling* (branching on perception failure, retry with modified parameters, primitive or pose fallbacks, precondition and outcome checks, safe degradation), *strategic_coherence* (locate-approach-grasp-transport-place decomposition, physical-prerequisite ordering, intermediate-result propagation), *physical_affordance_grounding* (grasp poses based on object geometry, placement on support surfaces, reachability and collision checks), and *adaptive_exploration_search* (multiple candidate objects or poses, search-space expansion after inconclusive observation, strategy change based on intermediate results). The judge additionally returns `integrity_hardcoding_flag` with severity in $\{\text{NONE}, \text{MINOR}, \text{MAJOR}, \text{SEVERE}\}$; MAJOR caps all five axis scores at 0.25 and SEVERE sets them to 0.0. None of the judge channels enter the acceptance rule of Eq. (1) in the main text; they are reflection-only side information used to drive the next mutation prompt. The verbatim rubric (`judge_prompt.md`) is released in the supplementary submission artifact.

A.1.3 RAI Portability Protocol

RAI is RobotecAI's open-source ROS 2-oriented embodied-agent framework [17], evaluated here on an O3DE manipulation benchmark. RHO evolves the prompts and tool-edit Python module that adapt RAI's tools for that benchmark; it does not evolve RAI's deployment loop, ROS 2 connector layer, or inference-time driver.

The benchmark uses class-stratified train/test splits: $21/21$ on the easy pool and $22/21$ on the hard pool. RHO selects the best-by-train candidate after ten generations per cell, then evaluates the held-out test half; the easy headline averages $n{=}21$ held-out test tasks over three runs. The easy O3DE multi-file cell lifts the seed from **0.683** to **0.889** ($+0.206$, $p < 0.001$), and the hard multi-file cell lifts the seed from **0.235** to **0.443** ($+0.208$, $p < 0.001$).

At deployment, HELIX's training-time mutator is not invoked, but RAI's own deployment driver remains on the inference path. The CaP-X no-solver-owned code-generation invariant remains a property of the CaP-X solver instantiation; RAI tests portability of the train-time edit-and-freeze recipe to a ROS 2 embodied-agent substrate.

The RAI replication ran on a single AMD Ryzen AI Max+ 395 workstation with 128 GB unified memory, with the RAI agent, ROS 2 stack, and O3DE simulator co-resident on-device. RAI deployment numbers are reported in Section A.3.

A.1.4 Mutator reflection-prompt template

The reflection prompt issued to the coding agent at each mutation step is assembled by the HELIX runtime from (i) a fixed task-instruction preamble, (ii) the parent candidate's full evaluator output, (iii) the `background` field of `helix.toml` (the domain-context scaffold), and (iv) a fixed task request. The skeleton below was captured verbatim from a representative mutation step in the canonical Skeleton run; placeholder fields are bracketed. The Diagnostics block is dynamic: its field set is determined by whatever per-instance `side_info` keys the user's evaluator emits, rendered in sorted-key order, so the fields shown below reflect the canonical run's evaluator and may differ for other evaluators.

```
Task instructions:
- Work directly on the requested code changes using the workspace files.
- Do not request confirmation or clarification; choose a reasonable approach
and continue.
- If one approach fails, try an alternative and keep progressing.
- Use available tools to inspect, edit, and validate changes.

## Objective
Maximize the environment-provided shaped reward for simulated Franka
manipulation tasks. When evaluation diagnostics are present, prioritize
the concrete examples shown there and preserve behavior that already
scores well. Maintain the solver.main.run(ctx) entry point and use only
the documented runtime API.

## Current Evaluation Scores
success: <parent_score>

## Diagnostics
### Example <task_family>__<trial_id>
#### diagnostic_focus
<lowest_reward_failed_for_vdm | highest_reward_succeeded | mixed>
#### evaluation_diagnostics
##### capability_scores
###### state_uncertainty_management        <float in [0, 1]>
###### robustness_contingency_handling     <float in [0, 1]>
###### strategic_coherence                 <float in [0, 1]>
###### physical_affordance_grounding       <float in [0, 1]>
###### adaptive_exploration_search         <float in [0, 1]>
##### task_failure_hypotheses              <up to 3 items>
##### minimal_next_experiments             <up to 3 items>
##### preserve_existing_behavior           <up to 3 items>
##### api_misuse_observations              <0+ items, file:line cited>
##### integrity_warning
###### severity                            <none | minor | major>
###### evidence                            <up to 3 items, file:line cited>
###### score_caps_applied                  <none | applied cap>
#### task_completed                        <True | False>
#### top_k_best_example_history            <top-K accepted ancestors by score>
#### visual_difference_summary             <vdm free-text prev/current
↪  narration>
  [... one block per sampled instance; some carry only task_completed
    + top_k_best_example_history ...]

## Background / Context
  [Domain-context scaffold from helix.toml: API_REFERENCE.md pointer,
    documented runtime primitives, closed-loop FSM preference, integrity
    rules, observable verification gates.]

## Your Task
```

```
Analyse the evaluation results above and improve the code to better
achieve the objective.  Make targeted, meaningful changes.  You may
read, edit, create, or delete files as needed.

## Turn Budget
You have a 150-turn limit for this task, where turns refer to how many
tool calls or interactions you can make.  Plan your work accordingly:
prioritize the highest-impact changes first and be efficient with your
tool usage.
```

The capability-scores block carries the LLM-judge axes defined in Appendix A.1.2; the verbatim judge prompt (`judge_prompt.md`) is released in the supplementary submission artifact.

A.1.5 Seed configurations

HELIX requires a generation-0 seed repository that exposes the benchmark's invocation signature: we do not tell the agent which benchmark it is solving, so, without the signature, the mutator would have to guess how its program should be invoked. We compare two configurations.

Skeleton seed (canonical Robosuite run). A coding agent pre-populates a multi-file Python package with typed stubs for the task router, per-task solvers, and shared helpers (initial source-line counts in parentheses):

```
solver/
__init__.py        (1L)   from solver.main import run
main.py            (27L)  prompt-keyword router -> 6 per-task solvers
common.py          (30L)  prompt_text, observe, first_camera, note_status
tasks/
__init__.py      (0L)
cubes.py         (stub) solve_cube_lifting, solve_cube_relation
nut_assembly.py  (stub) solve_nut_assembly
spill_wipe.py    (stub) solve_spill_wipe
two_arm.py       (stub) solve_two_arm_lift, solve_two_arm_handover
```

The 200-generation canonical Codex GPT-5.5 Multi-file run selects `g134-s134` at **70.0%** (490/700) on CaP-Bench Robosuite S4 external evaluation.

Stub-main seed. A minimal alternative supplies only the entry point and no task scaffolding:

```
solver/
__init__.py        (1L)   from solver.main import run
main.py            (6L)   def run(ctx): prints + returns {"status": "stub"}
```

Under the stub-main seed, the loop reaches generation 250 and collapses everything into a single monolithic `main` function: under the same task instructions and tool surface as the Skeleton run sees, the mutator rarely refactors this structure, and the run terminates without a candidate matching the Skeleton lineage's selected solver. This is the failure mode summarized in Section 3.1: with no task-family scaffold to modify, each mutation collapses into a single-function rewrite that the next mutation must again repair.

A.2 Experiments Appendix

A.2.1 Benchmark protocols

CaP-Bench is the seven-task Robosuite slice of CaP-X [4]: 100 trials per task, 700 trials total, evaluated under RHO's held-out S4 surface. **LIBERO-PRO** [5] is a perturbation benchmark of six conditions, defined by (object/goal/spatial)×(swap/task), each with 10 tasks and 50 trials.

A.2.2 Deployment audit and S4 contract

RHO training runs in the M4 tier: multi-turn execution, VDM access, and a coding-agent loop that can inspect traces and mutate code. All reported CaP-Bench and LIBERO-PRO deployment numbers are S4 single-turn runs with VDM disabled and no solver-owned LLM code-generation client. For the canonical $490/700$ CaP-X result, the S4 protocol is recorded in the provenance manifest. The VDM=0 / single-turn evaluation is the CaP-X S4 contract. A source-level audit finds no deployed-solver imports of `openai`, `anthropic`, `google.genai`, `litellm`, `requests`, `httpx`, or the CaP-X sidecar transport.

A.2.3 Comparator reconstruction

On CaP-Bench, the comparator is Agent0 in the +3M configuration (68.29% on Fig. 8; 9 candidate code-generation queries, an ensemble synthesis call, and a VDM call per turn; [4]). On LIBERO-PRO, the comparators are the VLA and CaP-Agent0 rows from CaP-X and the LIBERO-PRO paper: OpenVLA scores 0.00 on all six aggregate cells, and CaP-Agent0's 18.17% ($545/3000$) is a derived aggregate over six equally weighted swap/task cells, not a published headline [4, 5]. The LIBERO-PRO comparison excludes LIBERO-Plus aggregates [10]. Among LIBERO-trained baselines, $\pi_{0.5}$ [8] concentrates almost all of its success on the swap cells (0.38 goal-swap, 0.17 object-swap, 0.20 spatial-swap) and collapses to near-zero on the task cells (0.00 goal-task, 0.01 object-task, 0.01 spatial-task) [5].

A.2.4 Runtime and latency ledger

HELIX deployment wall-clock time averages 189.1 **s/trial** (median 161.5, p95 438.5) across the 700 CaP-Bench trials (7 tasks $\times$ 100 trials), measured on a dual-socket Intel Xeon Platinum 8570 host with two NVIDIA B200 GPUs. This is full per-episode wall-clock time: it includes simulator, robot, perception, and logging, not solver-owned code-generation overhead alone, and Agent0 would share this floor. Table 5 resolves the aggregate into per-task means: `cube_stack` anchors the floor at 44.8 s, while `nut_assembly` (409.2 s mean) drives the p95 tail almost on its own. The source audit finds no runtime retrieval index, embedding store, or skill-selection API in the deployed solver: the zero-call result is a repository-as-library deployment, not a retrieval-backed runtime agent.

The structural call-count contrast does not support a defensible dollar inequality: CaP-X discloses none of Agent0's per-call token mix, retry distribution, cache behavior, or VDM pricing. Figure 8 maps where that compute lives: Agent0 issues up to 66 deploy-time LLM calls per trial, while RHO issues no LLM code-generation calls in the control loop, at comparable CaP-Bench accuracy.

A.2.5 Backend selected-run sensitivity

Mutator harness configurations. Two coding-agent harnesses drive the backends in this paper. Codex backends use OpenAI's Codex CLI with `gpt-5.5` at reasoning effort `xhigh`. Anthropic backends use Anthropic's Claude Code with `claude-sonnet-4-6` (Sonnet 4.6 rows, including Sonnet 4.6 Behavior Tree) and `claude-opus-4-7` (Opus 4.7 rows, including Opus). The Qwen rows in Tables 2 and 3 use Claude Code redirected via `ANTHROPIC_BASE_URL` to a local vLLM endpoint serving `Qwen3.6-27B` (tensor parallel 2, context window $262{,}144$, `qwen3_coder` tool-call parser). Both harnesses expose comparable tool surfaces (file edits, shell execution, environment interaction). HELIX records per-mutation token usage uniformly across them.

Table 5: Per-task deploy wall-clock for HELIX on CaP-Bench. Per-task means span a 9.1× range: `cube_stack` anchors the floor at 44.8 s, while `nut_assembly` at 409.2 s drives the aggregate p95 (438.5 s) almost alone, its median (407.9 s) already exceeding that aggregate p95. Across all 700 trials the mean is 189.1 s and the median 161.5 s. Each per-trial figure is full per-episode wall-clock (environment reset, perception, simulation, logging) for the selected `g134-s134` solver on the S4 oracle path; no LLM code-generation call is issued inside the trial (host-provided perception primitives such as SAM 3 and Molmo still run).

Task	mean (s)	median (s)	p95 (s)	n
`cube_lifting`	131.3	104.3	282.6	100
`cube_stack`	44.8	46.0	60.5	100
`cube_restack`	106.7	80.1	237.5	100
`nut_assembly`	409.2	407.9	514.7	100
`spill_wipe`	309.5	309.5	385.2	100
`two_arm_lift`	168.1	173.1	234.1	100
`two_arm_handover`	153.8	171.1	186.1	100
All 7 tasks	**189.1**	**161.5**	**438.5**	**700**

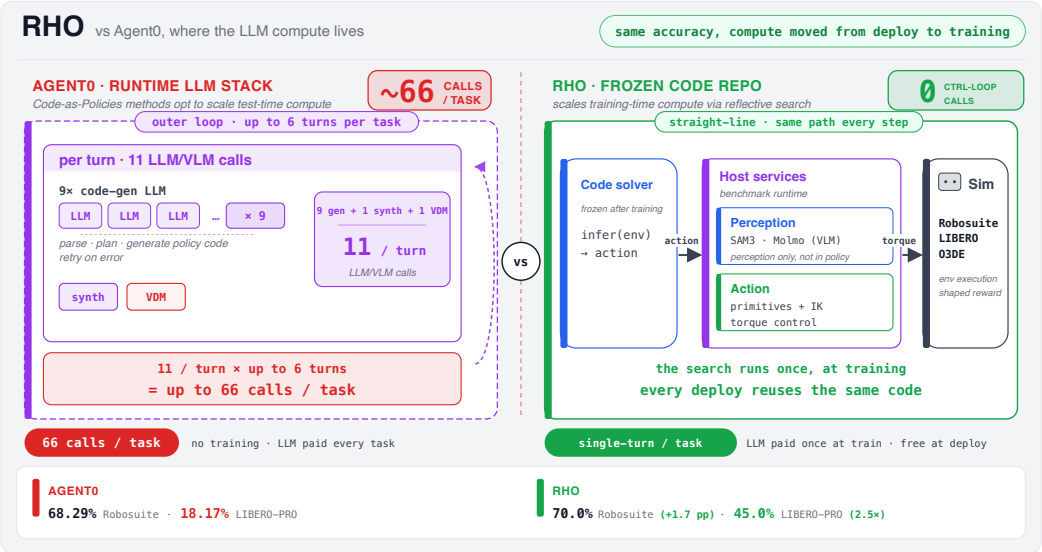


Figure 8: Where the LLM code-generation compute lives. Agent0 issues up to sixty-six LLM calls per trial at deploy time (nine code-generation queries, one ensemble synthesis call, and one VDM call per turn, over up to six turns); RHO issues no LLM code-generation calls in the control loop, having amortised that compute at train time. Both reach comparable CaP-Bench accuracy (68.29% Agent0 vs **70.0**% RHO) on the same host-provided perception and action primitives (SAM 3, Molmo, Contact-GraspNet).

Selected-run headline spread. Across accepted single CaP-Bench S4 runs, the selected-run headline varies substantially: the reported Codex-backed run reaches 70.0%, an alternative Codex configuration reaches 63.9%, Opus reaches 61.9%, Qwen3.6-27B reaches 35.0%, and Sonnet 4.6 Behavior Tree reaches 33.9%, a 36.1 pp spread between the highest and lowest accepted backend. The entries are selected-run comparisons, not a multi-seed backend ranking.

A.2.6 Training-time REaaS tool use

The reported run records 1,776 calls to the robot environment-as-a-service (REaaS), evidence that the mutation agents used the environment tools throughout the search rather than emitting code zero-shot.

The REaaS. During a mutation the coding agent may exercise five REaaS endpoints. `reset` starts a task trial and returns the task prompt, the allowed primitive calls, and a summarized initial observation; `observe` returns the current observation; `call` invokes a single perception or action primitive from the allowed set (for example, text-prompted segmentation or grasp planning); `run` executes the whole candidate solver end-to-end; and `vdm` poses a natural-language question to a vision-language model that performs a visual difference between the current observation and the preceding timestep, reporting what changed. The first is per-trial setup; the rest are how the agent probes and reads the live environment.

Environment engagement, not tool volume, tracks deployed quality. Four of the five service channels in Table 4 fall monotonically down the success column. From the 70.0% solver to the 18.1% one, the mutator runs candidates $576 \to 486 \to 405 \to 61 \to 11 \to 5$ times, invokes control and perception primitives $293 \to 247 \to 182 \to 32 \to 3 \to 0$ times, queries the visual-difference module $166 \to 86 \to 84 \to 8 \to 0 \to 0$ times, and re-observes the scene $43 \to 31 \to 31 \to 8 \to 0 \to 0$ times. Only `reset`, the per-trial environment setup, fails to track score.

Call volume does not track quality; grounded depth does. The FSM run issues more total service calls than the Behavior Tree run (62 versus 28) yet deploys the weakest solver, because 57 of its 62 calls are resets: it resets the simulator 57 times, runs a candidate 5 times, and never once queries the visual-difference module or re-observes a scene. A search that resets but never observes cannot distinguish a working candidate from a broken one.

Grounded search finds the verification that works. A visual-difference query lets the mutator see the gap between what a candidate intended and what it achieved, and a re-observation confirms the world changed, so a search that exercises both retains correctness mechanisms a blind search cannot score. The dimension-wise artifact analysis shows why: the feedback-heavy winner is the only solver whose verification distrusts its own perception, cross-checking text-prompt against point-prompt detections and flagging a stale, unchanged scene. It is also the only one whose terminal check triggers a corrective re-execution rather than logging a status string. The Qwen FSM solver carries the *densest* verification coverage of the four artifacts, yet it never grounds that verification against visual difference during search and narrates failure without repairing it. More verification code does not help; grounding the search so the mutator finds verification that *works* does.

Reaching for visual feedback is a frontier-model behavior. The same channels separate the backends by mutator scale, not only by score. Only the two frontier mutators ground their search at volume: Codex (`gpt-5.5`) issues 166 visual-difference and 43 re-observe queries in the multi-file run and 86 and 31 in the single-file run, and Claude Code (`Opus 4.7`) issues 84 and 31. The smaller Sonnet 4.6 issues none of either, and open-source Qwen3.6-27B issues only 8 visual-difference and 8 re-observe queries in free-form search and none under the FSM constraint, one to two orders of magnitude below the frontier mutators. Mutator scale, depth of grounded search, and final success co-vary across these single selected runs, so we do not claim one drives another; the consistent pattern is that the frontier mutators which ground their search most deeply are the ones that ship the robust $70.0\%/63.9\%/61.9\%$ solvers.

A.2.7 LIBERO-PRO condition cells

RHO on LIBERO reaches 45.0% on LIBERO-PRO ($1{,}351/3{,}000$) with no LIBERO gradient update. The selected candidate is `g32-s32` at generation 63 of the reflective-evolution lineage (an initial 31-generation lane through `g31-s31` plus a 32-generation warm-restart lane re-seeded from `g31-s31`). Its six condition scores are 50.6% goal-swap, 55.6% goal-task, 12.2% object-swap, 37.2% object-task, 53.0% spatial-swap, and 61.6% spatial-task. Tables 6, 7, and 8 give the per-task breakdown of these cells against the CaP-X baselines (OpenVLA, π_0, $\pi_{0.5}$, CaP-Agent0), reproducing CaP-X Tables 6–8 with an added RHO column. Throughout, Pos is the initial-position perturbation (our `swap` cell) and Task is the instruction perturbation (our `task` cell); values are the fraction of 50 trials solved.

Table 6: Per-task success (fraction of 50 trials) on `libero-object`; baselines from CaP-X [4, 5]. The two VLAs transfer almost nothing (≈ 0 on every task), and $\pi_{0.5}$ is bimodal: near-perfect on a few memorized objects under position swaps (bbq_sauce 1.00, butter 0.54) yet ≈ 0 elsewhere and on all instruction swaps, so neither baseline generalizes. The two code-as-policies agents instead spread partial success across objects. RHO is the most language-robust (Task 0.37 vs Agent0's 0.18, as its perception re-grounds the renamed target), but object-swap is its one losing cell (Pos 0.12 vs 0.22): the mutator-added, hand-fitted grasp schedule fails to transfer when the object is physically relocated (analyzed below). Best column average is in bold; *Place(obj, basket)*: pick up *obj* and place in the basket.

	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0		**RHO**	
Task (symbolic form)	Pos	Task	Pos	Task	Pos	Task	Pos	Task	Pos	Task
Place(alphabet_soup, basket)	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.04	0.12	0.40
Place(bbq_sauce, basket)	0.00	0.00	0.00	0.00	1.00	0.02	0.12	0.42	0.32	0.88
Place(butter, basket)	0.00	0.00	0.00	0.00	0.54	0.00	0.26	0.18	0.12	0.98
Place(chocolate_pudding, basket)	0.00	0.00	0.00	0.00	0.00	0.02	0.18	0.48	0.00	0.00
Place(cream_cheese, basket)	0.00	0.00	0.10	0.00	0.00	0.00	0.12	0.06	0.00	0.00
Place(ketchup, basket)	0.00	0.00	0.00	0.00	0.20	0.02	0.32	0.12	0.00	0.04
Place(milk, basket)	0.00	0.00	0.00	0.00	0.00	0.00	0.38	0.02	0.18	0.26
Place(orange_juice, basket)	0.00	0.00	0.00	0.00	0.00	0.02	0.30	0.02	0.34	0.40
Place(salad_dressing, basket)	0.00	0.00	0.10	0.00	0.00	0.00	0.32	0.00	0.14	0.76
Place(tomato_sauce, basket)	0.00	0.00	0.00	0.00	0.00	0.00	0.16	0.48	0.00	0.00
Average	0.00	0.00	0.00	0.00	0.17	0.01	**0.218**	0.182	0.122	**0.372**

Table 7: Per-task success (fraction of 50 trials) on `libero-goal`; baselines from CaP-X [4, 5]. The VLAs again collapse and $\pi_{0.5}$ only spikes on a handful of memorized positions (a few Pos cells near 1.0, Task ≈ 0). RHO is the only system with broad two-sided competence: it partially solves most goal tasks under both relocated objects (Pos 0.51) and reworded goals (Task 0.56), roughly doubling Agent0 (0.26/0.17) by being consistently capable rather than winning isolated cells. Best column average is in bold; actions *Open*/*Put*/*Push*/*TurnOn* follow CaP-X notation.

	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0		**RHO**	
Task (symbolic form)	Pos	Task	Pos	Task	Pos	Task	Pos	Task	Pos	Task
Open(cabinet, drawer_mid)	0.00	0.00	0.00	0.00	0.00	0.04	0.00	0.00	0.00	0.00
Put(bowl, drawer_top)	0.00	0.00	0.00	0.00	0.94	0.02	0.04	0.00	0.30	0.32
Push(plate, stove_front)	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.10	0.00	0.00
Put(bowl, plate)	0.00	0.00	0.00	0.00	0.00	0.02	0.36	0.38	0.98	0.98
Put(bowl, stove)	0.00	0.00	0.00	0.00	0.00	0.04	0.22	0.12	1.00	0.98
Put(bowl, cabinet_top)	0.00	0.00	0.00	0.00	0.00	0.02	0.60	0.04	0.98	0.98
Put(cream_cheese, bowl)	0.00	0.00	0.00	0.00	0.98	0.02	0.04	0.34	0.10	0.10
Put(wine_bottle, rack)	0.00	0.00	0.00	0.00	0.88	0.02	0.02	0.12	0.58	0.62
Put(wine_bottle, cabinet_top)	0.00	0.00	0.00	0.00	0.98	0.02	0.62	0.40	0.64	0.64
TurnOn(stove)	0.00	0.00	0.00	0.00	0.00	0.00	0.66	0.18	0.48	0.94
Average	0.00	0.00	0.00	0.00	0.38	0.00	0.256	0.168	**0.506**	**0.556**

Object-swap failure analysis. `object-swap` is the only LIBERO-PRO cell that perturbs scene composition: the BDDL relocates the named target object from its training-distribution slot (`target_object_region`, $y \approx -0.24$) to a peripheral slot (`other_object_region_4`, $y \approx -0.08$), and a randomized distractor fills the original slot. Perception is not the failure mode: Molmo and SAM still localize the relocated target at 0.94+ mask confidence. Of 439 failed trials, 325 (74%) involve **false-positive lift verification**: after a missed grasp, the solver's `verify_grasp_lift` routine (added during evolution) re-segments the source label and latches onto either the new distractor or noise near the gripper with a score of ≈ 0.02, returning `lifted` despite empty fingers,

Table 8: Per-task success (fraction of 50 trials) on `libero-spatial`; baselines from CaP-X [4, 5]. This is RHO's strongest suite and its widest margin: explicit perception-driven grounding of spatial referents (`between`, `next_to`, `on`) holds up under both perturbations (Pos 0.53, Task 0.62), while Agent0 stays near the floor (0.12/0.14) and $\pi_{0.5}$ again only memorizes isolated positions (on(ramekin) 0.98, on(cabinet_wood) 0.90). Best column average is in bold; *Pick(src, plate)*: pick the black bowl from *src* and place it on the plate.

	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0		**RHO**	
Task (symbolic form)	Pos	Task	Pos	Task	Pos	Task	Pos	Task	Pos	Task
Pick(between(plate,ramekin), plate)	0.00	0.00	0.00	0.00	0.02	0.00	0.22	0.14	0.92	0.60
Pick(table_center, plate)	0.00	0.00	0.00	0.00	0.00	0.02	0.22	0.14	0.96	0.96
Pick(drawer_top(cabinet_wood), plate)	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.10	0.06	0.04
Pick(next_to(cookie_box), plate)	0.00	0.00	0.00	0.00	0.00	0.02	0.00	0.10	0.00	0.28
Pick(next_to(plate), plate)	0.00	0.00	0.00	0.00	0.00	0.00	0.10	0.20	0.56	0.64
Pick(next_to(ramekin), plate)	0.00	0.00	0.00	0.00	0.12	0.02	0.30	0.14	0.78	0.76
Pick(on(cookie_box), plate)	0.00	0.00	0.00	0.00	0.00	0.00	0.14	0.08	0.10	0.90
Pick(on(ramekin), plate)	0.00	0.00	0.00	0.00	0.98	0.02	0.02	0.20	0.64	0.42
Pick(on(stove), plate)	0.00	0.00	0.00	0.00	0.02	0.00	0.08	0.14	0.86	0.98
Pick(on(cabinet_wood), plate)	0.00	0.00	0.00	0.00	0.90	0.00	0.08	0.16	0.42	0.58
Average	0.00	0.00	0.00	0.00	0.20	0.01	0.118	0.14	**0.530**	**0.616**

and the subsequent place step releases nothing. The remaining 114 (26%) exhaust the 12-attempt grasp budget on a hand-authored grasp-pose schedule (`container_side_yneg/ypos/xpos/xneg`, `center_yaw0/90`) whose offsets and `close_depth` defaults were calibrated for the on-distribution slot. Per-subtask success rates split cleanly by whether the solver carries a per-object branch: 0% on `cream_cheese`, `ketchup`, `tomato_sauce`, `chocolate_pudding` (no branch), 32–34% on `bbq_sauce` and `orange_juice` (hand-coded variants), and 12–18% on the remaining categories. CaP-Bench exposes a Contact-GraspNet `plan_grasp` primitive that the selected solver does call: at the swapped pose, its confidence drops from ≈ 0.8 to ≈ 0.2, but it still returns liftable candidates. The mutator elaborated the solver from 427 lines at generation 2 to 2,016 lines at generation 32, accumulating roughly 53 lines per generation in per-category branches and manual-schedule entries. This overlay is what fails on object-swap.

Warm-restart configuration changes. The initial lane set:

- acceptance_criterion = strict_improvement
- val_stage_size unset
- num_sampled_groups unset
- num_examples_per_group unset

The warm-restart lane was changed to:

- acceptance_criterion = improvement_or_equal
- val_stage_size = 45
- num_sampled_groups = 2
- num_examples_per_group = 1

Each setting in the second configuration matches an addition in Section 3.1.

A.2.8 Training ledger

The reported run's persisted `state.budget` ledger records 6,814 evaluations, 625,276,104 input tokens, 88,851,968 cached input tokens, and 3,144,685 output tokens; summed over the 87 retained

per-worktree `.helix_backend_result.json` files, total backend usage is 386,757,173 input tokens, 373,486,336 cached input tokens, and 1,757,060 output tokens. At GPT-5.5 API pricing ($5/M input, $0.50/M cached input, $30/M output), the canonical `state.budget` tokens correspond to an estimated training cost of $2,821: $2,682 for 536.4M uncached input tokens, $44 for 88.9M cached input tokens, and $94 for 3.14M output tokens. The persisted `state.json` records `cost_usd=0.0`; the orchestrator does not track API costs directly. The estimate is derived from the token counts and public pricing as of May 2026. End-to-end wall-clock for the RHO training run was 79.4 hours ($\sim$3 days 7 hours) on a dual-socket Intel Xeon Platinum 8570 host with two NVIDIA B200 GPUs, measured from the `.helix/config.toml` creation time to the final `.helix/state.json` write. The persisted run completed under `frontier_type=instance` and `max_generations=200`, reaching `generation=200` and `mutation_counter=200`. The re-parse finds 1,776 REaaS calls across 87 retained HELIX worktree transcripts: 698 reset, 576 run, 293 call, 166 vdm, and 43 observe. Across the 87 retained frontier entries, `lineage.json` labels 86 nodes as mutations and 1 as the seed; the original four-way mutation taxonomy is not reliably recoverable from these artifacts. Aggregated Python source diffs over the 87 frontier worktrees against the seed total $+153{,}519/-1{,}730$ lines, with the selected solver introducing 74 net new Python function definitions over the baseline. The 200 mutation slots resolved into 87 retained frontier members (86 accepted mutations and the generation-0 seed), 77 rejected at the training gate, 28 skipped because the parent already attained a perfect subsample score, and 8 mutator-step failures attributable to host-side intermittent issues and OpenAI API congestion (HTTP 4xx). Judge-model and visual-difference-model API costs are not included in the $2,821 estimate above. The sandbox's integrity controls fired zero times across the canonical run: no candidate triggered protected-file tamper rejection or an LLM-as-a-judge integrity flag of MAJOR or SEVERE severity (Appendix A.1.2). Two LIBERO-PRO perturbation cells remain weak (object-swap at 12.2%, object-task at 37.2%); the object-swap collapse traces to mutator-added overlays in the selected solver (a hand-fitted grasp schedule and a re-segmentation-based lift verifier) rather than perception-primitive degradation (Appendix A.2.7, with the design implication discussed in Section 7).

A.2.9 Cross-run training budget

Table 4 reports per-run token totals and dollar estimates for the reported RHO run, the four accepted backend-sensitivity alternates, and four first-mutation single-shot baselines (one per base model: GPT-5.5, Opus, Sonnet, Qwen; the GPT-5.5 baseline serves both Codex configurations they share). CaP-Agent0 [4], $\pi_{0.5}$ [8], and OpenVLA [7] do not publish the per-run token/dollar ledger this comparison requires, so we omit them. Accepted alternates range from $\sim$$1,282 (Opus, aggregated upstream chain) down to $124.44 (Sonnet 4.6 Behavior Tree, state recorded). The $732 Codex GPT-5.5 Single-file alt-config figure covers only the 150-generation warm-restart segment whose tokens `state.json` records; the $1,665 single-file cost in Table 3 is the cumulative total for the same lineage across its 100-generation parent and 150-generation warm-restart. All single-shot baselines and every evolved RHO run start from the same Skeleton seed (Appendix A.1.5); their costs and held-out scores appear in Table 2 in the main text. The strongest single-shot point (Codex GPT-5.5 at $210/700 = 30\%$) is the reference cited in §A.2.8. Against the full single-shot range of $0–$3.19 per pass and 13–30% on the same split, the reported RHO run pays $2,821 once, reaches $490/700$, and then issues 0 LLM code-generation calls per control step (§A.2.8).

A.2.10 Evaluator integrity details and unmeasured variants

For the reported run, accepted candidates were generated under protected-file manifest checks and evaluator-internal omission. The selected solver was replayed under the VDM-disabled S4 harness. These checks support the training-time evaluator-integrity claim; sandbox-off and manifest-only variants are not measured in this submission.

A.3 RHO on an LLM-in-the-loop Robotics Stack

To test whether RHO transfers beyond the primitive-only CaP-Bench surface, we instantiate it with HELIX, REaaS, and evaluator isolation on Robotec's RAI [17] `manipulation_o3de` benchmark. The deployment loop here *includes* an LLM, so the candidate repository defines the prompting and tool surface that the deployed LLM consumes rather than a Python controller. The same evolutionary recipe and acceptance gate are reused; the RAI run changes the candidate artifact and adds a fixed tool-name contract.

A.3.1 Background: the RAI framework and the `manipulation_o3de` benchmark

RAI [17] is an open-source agent framework released by Robotec.AI for connecting off-the-shelf LLMs to physical robots. Unlike many LLM-agent benchmarks, RAI is constructed directly on top of the ROS 2 middleware that production robotics systems already use: agents talk to perception, planning, and control nodes over the same topics and services that real robot software speaks. This makes it a useful stress test for industrial settings, where the value proposition of an LLM agent is precisely that it can drive the existing ROS 2 interfaces a robotics team has already deployed.

RAI ships a benchmark suite, `rai_bench`, whose `manipulation_o3de` track exercises a tabletop manipulator simulated end-to-end with O3DE (Open 3D Engine) for rendering and physics, ROS 2 + MoveIt for control, and Grounding DINO + SAM 2 [99] for open-vocabulary perception. The agent is given three fixed tools, `get_object_positions` (perception query by name), `get_ros2_camera_image` (vision), and `move_to_point` (motion: a `grab` or `drop` at a 3-D coordinate), and is asked to accomplish a natural-language manipulation task. An automated scorer checks the final object configuration against per-task tolerance windows. Throughout, the LLM is exposed to exactly the same ROS 2 surface that one would expose on a physical robot; nothing about the agent interface is simulation-specific.

Task classes. The benchmark defines five task classes: PLACEOBJECTATCOORD (move a named object to a specified (x, y) target), MOVEOBJECTSTOLEFT (move all instances of a category to the left half of the table), PLACECUBES (place colored cubes adjacent to each other), BUILDCUBETOWER (stack cubes vertically), and GROUPOBJECTS (cluster objects of the same category together). Each class is instantiated at several difficulty levels that vary the number of distractor objects, the tightness of the success tolerance window, and whether initial conditions include pre-stacked or pre-clustered objects. Figure 9 shows a representative held-out trial of each class.

Difficulty levels. The benchmark exposes five difficulty bands (TRIVIAL, EASY, MEDIUM, HARD, VERY_HARD) that adjust three independent levers in concert:

- *Scene clutter.* TRIVIAL scenes contain a single object on the tabletop; EASY 2–3 objects; MEDIUM 4–6; HARD and VERY_HARD up to a dozen objects. Several HARD scenes additionally carry a `_stacked` flag, meaning the scene boots with cubes already vertically arranged, so the agent must plan around existing structure rather than building from scratch.
- *Task statement.* MOVEOBJECTSTOLEFT and GROUPOBJECTS progress from single-type queries ("move all carrots left", "cluster the apples") at TRIVIAL/EASY to multi-type combinations ("move carrots, apples, and yellow cubes left") at MEDIUM and above. BUILDCUBETOWER starts at three colors and ramps toward single- or two-color towers (forcing the agent to stack identical-looking cubes precisely) at VERY_HARD.
- *Pass criteria.* Several classes tighten their numerical success thresholds at higher levels. The cleanest example is PLACECUBES: EASY accepts cubes within $0.20\,\text{m}$ of a cluster partner, while MEDIUM and HARD halve this to $0.10\,\text{m}$ on much more cluttered scenes. PLACEOBJECTATCOORD runs only at the looser end of the spectrum (TRIVIAL and EASY) and is omitted from MEDIUM onwards entirely.

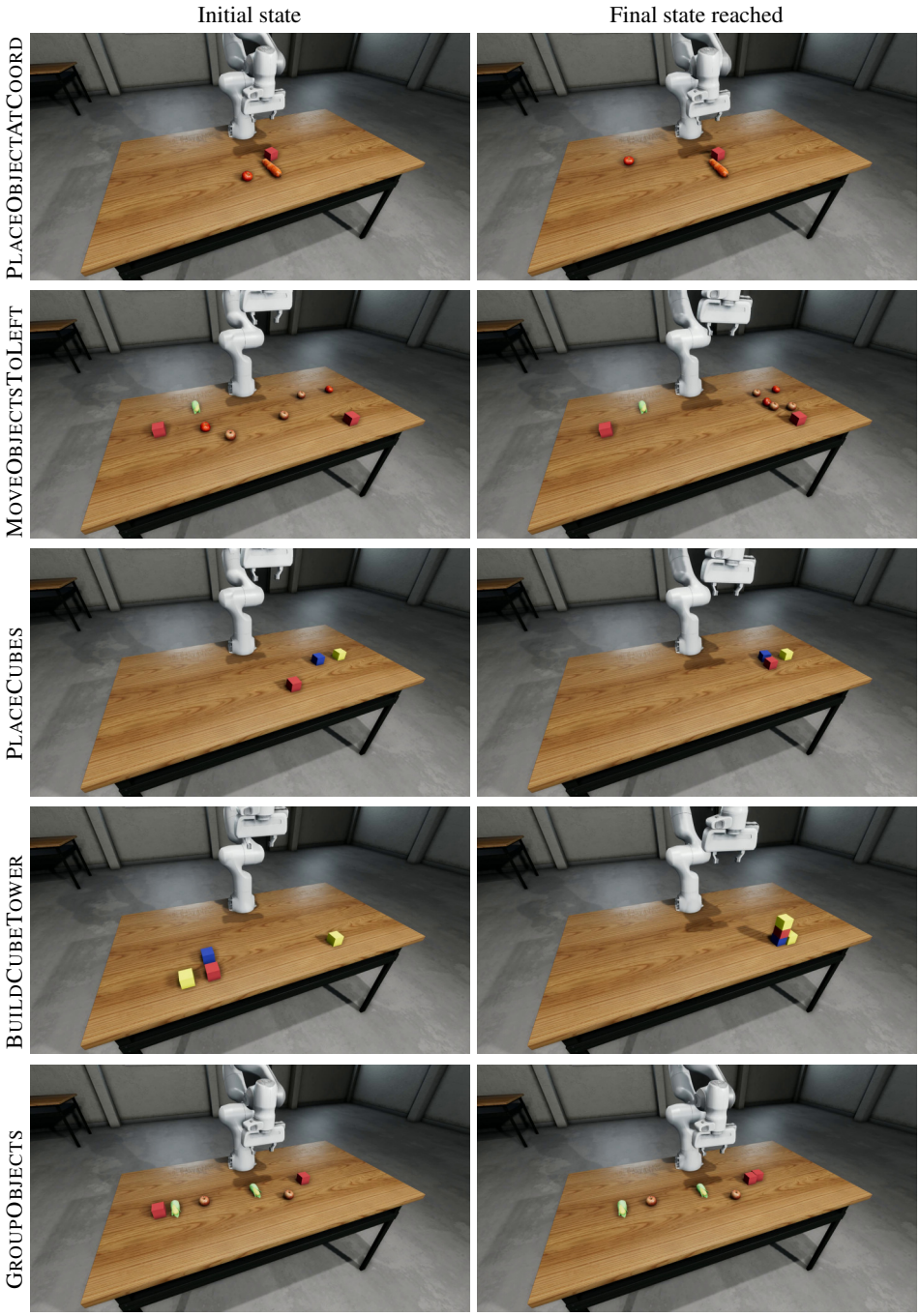


Figure 9: Representative held-out trials of the five RAI `manipulation_o3de` task classes (rows), each shown in its initial state (left) and the final state reached by an evolved RHO controller (right). PLACEOBJECTATCOORD: move one named object to a target (x, y) within a tolerance (all-or-nothing). MOVEOBJECTSTOLEFT: move every object of the requested type(s) to the $+y$ (left) half of the table while leaving others in place. PLACECUBES: arrange cubes so each is adjacent to at least one other cube. BUILDCUBETOWER: stack the requested-color cubes into a single vertical tower, excluding other object types. GROUPOBJECTS: gather objects into one connected, fully separated cluster per type. Except for PLACEOBJECTATCOORD, classes are scored by partial credit (the fraction of objects satisfying the goal).

Table 9: Task class availability across the five RAI `manipulation_o3de` difficulty bands. A check mark indicates that the benchmark's predefined task generator emits one or more scenarios of that class at that level. Several classes span three or four adjacent levels, with difficulty varying through scene clutter, task statements, and (for PLACECUBES) the numerical pass criterion.

Task class	TRIVIAL	EASY	MEDIUM	HARD	VERY_HARD
PLACEOBJECTATCOORD	✓	✓			
MOVEOBJECTSTOLEFT	✓	✓	✓	✓	
PLACECUBES		✓	✓	✓	
BUILDCUBETOWER			✓	✓	✓
GROUPOBJECTS			✓	✓	✓

Table 10: Per-difficulty class coverage and split sizes used in this section. EASY draws on the three classes that the bench instantiates at its looser end; HARD on the four classes that survive into the cluttered, tighter-criterion regime. Each cell uses a class-stratified 50/50 train/test split; the train half drives RHO evolution and the test half is evaluated only after evolution completes.

Task class	EASY (train/test)	HARD (train/test)
PLACEOBJECTATCOORD	15 / 15	–
MOVEOBJECTSTOLEFT	4 / 4	5 / 5
PLACECUBES	2 / 2	5 / 5
GROUPOBJECTS	–	9 / 8
BUILDCUBETOWER	–	3 / 3
Total	**21 / 21**	**22 / 21**

Per-difficulty class coverage. Not every task class is instantiated at every difficulty; the benchmark's task generator only emits a class at levels where it has tuned scene configurations and pass criteria for it. The classes that do appear overlap several adjacent levels rather than being level-exclusive (Table 9). MOVEOBJECTSTOLEFT, for example, appears at four of the five levels; what changes between them is the scene around it and the task statement, not the underlying scoring rubric.

For this study we package two of the bands into separate RHO cells, EASY and HARD. We chose these two rather than the strict extremes (TRIVIAL and VERY_HARD) because they expose the largest *within-band* diversity: each covers three or four task classes and spans both looser and tighter constraints within its respective regime, giving RHO a non-trivial mix of tasks to specialize against (Table 10).

EASY instances have one or two target objects, the loose 0.10–0.20 m pass criteria summarized above, and no stacking; the seed agent already solves a non-trivial fraction. HARD instances contain five or more objects with the tighter 0.10 m PLACECUBES criterion, include pre-stacked initial conditions, and introduce the two classes (GROUPOBJECTS, BUILDCUBETOWER) on which the seed agent is closest to total failure. Splitting evolution by difficulty allows RHO to specialize rather than average opposing per-class lifts and regressions inside a mixed bench.

A.3.2 Experimental setup

The headline configuration runs RHO with repository-level mutation (Section 3.1), allowing the mutator to edit the full repository (the agent's system prompt and the bodies of the three tools). We additionally run a single-file ablation that strips the mutator down to prompt edits only, recovering a GEPA-style [14] reflective optimizer, in order to isolate how much of the held-out lift is attributable to multi-file repository search versus prompt search alone.

Table 11: Held-out test scores on the RAI EASY subset ($n = 21$ tasks, averaged over 3 runs, GPT-5.1). MF is the headline repository-level mutation configuration; SF is the single-file ablation that restricts RHO to prompt edits. *Lift* is the within-run paired difference $\mathsf{best}_i - \mathsf{seed}_i$ averaged over runs; the t-statistic and 95% CI are computed on the resulting 63 per-task per-run paired differences. All values in percent.

Cell	Seed (mean $\pm$ std)	Best (mean $\pm$ std)	Lift	95% CI (t)
MF (headline)	68.3 ± 9.9	$\mathbf{88.9 \pm 2.8}$	$+20.6$ pp	$[+10.0, +32.0]$ pp ($t = 3.67$)
SF (ablation)	65.1 ± 7.3	$\mathbf{79.4 \pm 2.8}$	$+14.3$ pp	$[+4.0, +25.0]$ pp ($t = 2.61$)

- **Multi-file evolution (MF, headline).** RHO is allowed to mutate both the agent's system prompt (`solver/prompts.py`) and the *bodies* of the three tools (`solver/tools.py`), under a frozen tool-name contract enforced by an AST-level pre-commit check. A candidate that registers a new tool name, removes one, or changes any tool signature is rejected before it enters the frontier. This restricts the search space to changes that the LLM caller cannot directly observe, which is the most industry-realistic deployment scenario, since the agent's public tool API belongs to the surrounding production system and cannot be rewritten on a per-task basis.
- **Single-file evolution (SF, ablation).** RHO is restricted to mutating only the agent's system prompt (`solver/prompts.py`); the tool implementations are pinned to RAI's upstream wrappers around perception and motion. Comparing SF against MF on otherwise identical infrastructure isolates the contribution of multi-file repository search.

RHO runs for ten generations per cell on the train half, with a class-stratified minibatch size of three (one task per class per minibatch on EASY; one class rotated out per minibatch on HARD, which has four classes). The mutator is the `codex` CLI in sandbox mode, identical to that used in the canonical CaP-Bench Codex GPT-5.5 Multi-file run [4].

Held-out evaluation. For each cell we report seed (the unmodified upstream prompt and tools, re-evaluated independently in each cell over its own three runs, so the MF-cell and SF-cell seed means differ slightly because of sampler noise) and best-by-train (the candidate with the highest training-set mean across the evolution frontier) on the test half, which is never seen during evolution. To suppress single-run sampler noise, we average each cell over three independent runs and report mean $\pm$ std (in percentage points). Within-run paired lifts ($\mathsf{best}_i - \mathsf{seed}_i$) are tested across the resulting 63 per-task per-run pairs (21 tasks $\times$ 3 runs).

Hardware. All experiments in this section were run on a single workstation built around an **AMD Ryzen AI Max+ 395** APU with 128 GB of unified memory and an integrated Radeon 8060S GPU. The entire pipeline (ROS 2, MoveIt, the O3DE simulator with Vulkan rendering, Grounding DINO and SAM 2 perception, and the HELIX `codex` mutator) runs concurrently on this single APU. Wall-times reported below are end-to-end on this hardware.

A.3.3 Results: the easy subset

Table 11 summarizes held-out test performance on the EASY subset, averaged over three runs. The headline MF configuration lifts the test mean from **68.3%** to **88.9%** ($+20.6$ pp, $p < 0.001$ on the per-task paired t-test). The SF ablation, which strips HELIX's mutator down to prompt edits only, still produces a statistically significant lift (**65.1%** $\rightarrow$ **79.4%**, $+14.3$ pp, $p \approx 0.012$) but a smaller one: the MF best-by-train candidate is $+9.5$ pp above the SF best-by-train candidate in absolute test mean (95% CI $[+5.0, +14.0]$ pp, $t = 4.24$). The SF ablation is therefore sufficient to recover roughly $\frac{2}{3}$ of the MF lift on this benchmark; the remaining third is attributable to RHO's freedom to additionally edit the tool implementations.

Table 12: Held-out test compute totals across the 21 RAI EASY tasks (representative single run; the same sign and approximate magnitude hold under the 3-run average). Both the headline MF configuration and the SF ablation reach their higher accuracy with *fewer*, not more, tool calls and lower wall time than the seed.

Cell	Metric	Seed	Best	Δ
MF (headline)	Total tool calls	108	89	$\mathbf{-17.6\%}$
MF (headline)	Total wall time (s)	686	660	-3.8%
SF (ablation)	Total tool calls	98	89	-9.2%
SF (ablation)	Total wall time (s)	630	578	-8.2%

A useful side-channel diagnostic is the per-run variance of the evolved candidates: both MF and SF land at $\sigma = 2.8$ pp across the three runs, roughly $3\times$ tighter than the *seed* prompt's per-run variance on the same tasks ($\sigma = 9.9$ pp for the MF seed, $\sigma = 7.3$ pp for the SF seed). We attribute this to the evolved candidates narrowing the agent's effective action distribution, so the same OpenAI sampler perturbations translate into smaller score swings.

Compute: tool calls and wall time

Table 12 shows that the held-out accuracy lifts come with *lower*, not higher, inference cost. The headline MF candidate cuts tool calls by **17.6%** relative to its seed with a modest additional wall-time improvement. The SF ablation reaches a similar absolute tool-call count by editing the prompt alone (a **9.2%** reduction off a tighter seed), with an **8.2%** wall-time improvement.

The MF candidate achieves this efficiency *despite* its evolved `tools.py` having grown from 375 lines in the seed to 1,253 lines after evolution: the additional tool-side logic encodes more discriminating single calls (color verification, multi-query perception fallback, detection caching) rather than adding per-call overhead. The net effect is a more capable agent that, in addition, finishes each task in fewer round-trips, which matters wherever each tool invocation is a real hardware round-trip.

What RHO learned: insights from the evolved prompts and tools

The MF-evolved candidate (generation 8 of 10) is the natural artifact to inspect: it carries both the prompt-side and the tool-side edits RHO considered worth keeping. Its mutations fall into two layers.

The first is a structured rewrite of the seed prompt. RAI's stock manipulation system prompt (copied verbatim from `rai_bench.manipulation_o3de.interfaces.ManipulationTask`) is just six lines:

```
You are a robotic arm with interfaces to detect and manipulate
objects.
Here are the coordinates information:
x - front to back (positive is forward)
y - left to right (positive is right)
z - up to down (positive is up)
Before starting the task, make sure to grab the camera image to
understand the environment.
```

(Note the fourth line, which inverts the convention the benchmark actually uses for scoring. This is a single-token error in the upstream library that we deliberately kept in the seed so RHO would have to discover it on its own.) The MF-evolved candidate expands this into 78 lines of task-specific guidance, which the SF ablation arrives at independently in the same five recognizable categories:

1. **Tool-call discipline.** Explicit guidance to distinguish `get_ros2_camera_image` (no arguments) from `get_ros2_image` (`topic="/color_image5"`); a "perceive once, then act" loop that blocks the seed pattern of re-polling perception between every move.

2. **Coordinate-frame disambiguation.** The seed agent routinely treated $+y$ as *right* when the bench actually defines $+y$ as *left*. The evolved prompt states the convention up front as a hard rule.
3. **Object-name canonicalization.** Singular forms (`"apple"`, not `"apples"`), color-qualified cube names (`"red cube"`, `"blue cube"`), and an explicit dedup rule for cases where different color queries return overlapping centroids.
4. **Pick/place hard rules.** A three-step recipe (detect $\rightarrow$ grab at the exact returned centroid $\rightarrow$ drop), an explicit "no two grabs in a row" constraint, and a "use the detected z for tabletop drops" rule.
5. **Task-specific recipes.** One short recipe per task class in the train set, each ending with the meta-instruction "stop when satisfied; additional moves reduce score" (the benchmark penalises unnecessary actions).

To make these concrete, we give two excerpts from the MF-evolved prompt. The first is RHO's fix for the coordinate-frame bug above (category 2), now stated as a hard rule:

```
Coordinate frame:
- x is front/back on the table.
- y is left/right on the table.  Positive y is LEFT; negative y is
RIGHT.
- z is height.  Positive z is upward.
```

The second is one of the per-class task recipes (category 5), illustrating how concrete the evolved guidance becomes, right down to numerical stack heights tuned for the simulator's $\sim$5 cm cubes:

```
Single vertical tower:  collect detections for the requested cubes
and ignore absent colors.  If target cube centroids already share
the same x,y within about 0.03m with increasing z, stop.  Otherwise
choose one detected target cube or existing stack as the base and
keep its current x,y; do not invent a new tower center and never
use coordinate-task targets such as y=-0.4.  ...For each moved cube,
grab that cube at its listed centroid, then drop at the base x,y
with z values about 0.09, 0.145, 0.200, ...as the tower grows.
```

The second layer is the tool-body elaboration available only to MF, which the HELIX `codex` mutator was observed (via container traces) to develop iteratively by introspecting prior trace JSONs and per-task result summaries. The three most consequential tool-side additions are:

- **Color-mask verification on perception output.** After Grounding DINO and SAM 2 return candidate detections, the evolved tool samples each detection's segmentation mask in RGB and rejects detections whose dominant color does not match the requested color word (e.g. "yellow cube"). This eliminates the most frequent false-positive failure mode of the seed pipeline.
- **Multi-query perception fallback.** When a query such as "tomatoes" returns nothing, it automatically retries singular and color-qualified variants ("tomato", "red tomato") before giving up. This compensates for the open-vocabulary detector's known phrasing sensitivity.
- **Detection caching with grab-snap.** The last $\sim$80 detected centroids are cached. If the LLM calls `move_to_point(grab=...)` within $\sim$7.5 cm of a cached centroid, the tool snaps to the cached value. This defends against the LLM rounding off coordinates or accidentally substituting a drop target for a grab target.

The tool-side additions are precisely the kind of edits that the SF ablation cannot express: they live behind the agent's tool signatures rather than in front of them, and they fix failure modes (false-positive color detections, query phrasing sensitivity, coordinate rounding) that prompt rephrasing alone does not address. This drives the $+9.5$ pp absolute gap between the MF and SF best-by-train candidates in Table 11: the prompt-side edits raise the floor in both cells, but tool-side edits raise the

Table 13: Held-out test scores on the RAI HARD subset ($n = 21$ tasks, averaged over 3 runs, GPT-5.1). The per-task paired t-test spans 63 pairs (21 tasks $\times$ 3 runs). All values in percent.

Cell	Seed (mean $\pm$ std)	Best (mean $\pm$ std)	Lift	95% CI (t)
MF (headline)	23.5 ± 3.8	$\mathbf{44.3 \pm 2.4}$	$+20.8$ pp	$[+11.0, +31.0]$ pp ($t = 4.09$)
SF (ablation)	25.8 ± 12.0	$\mathbf{37.6 \pm 5.4}$	$+11.8$ pp	$[+2.0, +22.0]$ pp ($t = 2.33$)

Table 14: Per-class held-out test means on the RAI HARD subset (3-run averages, n shown is per-class task count summed over the three runs). The two cells cover different parts of the class space: SF recovers BUILDCUBETOWER and most of MOVEOBJECTSTOLEFT but regresses slightly on GROUPOBJECTS; MF essentially unlocks GROUPOBJECTS but does not move BUILDCUBETOWER. All values in percent.

Class	n (3 runs)	SF seed	SF best	MF seed	MF best
BUILDCUBETOWER	9	5.6	**44.4**	16.7	16.7
GROUPOBJECTS	24	15.3	9.7	0.0	**37.5**
MOVEOBJECTSTOLEFT	15	45.2	**76.5**	56.8	71.0
PLACECUBES	15	35.1	39.2	31.7	**45.0**

ceiling. This is the same pattern reported in Section 4 for the multi-file repository seed versus the single-file stub on CaP-Bench Robosuite.

The mutator independently identified the concrete failure modes (axis confusion, double-grabs, false-positive color detections, plural/singular query mismatches) and codified them into the prompt and tool wrappers automatically, end to end, on hardware available to a single engineer.

A.3.4 Results: the hard subset

The HARD subset is a substantially more demanding regime: scenes contain five to a dozen distractor objects, several start with pre-stacked cubes, the PLACECUBES pass criterion tightens to 0.10 m, and the four-class mix introduces GROUPOBJECTS and BUILDCUBETOWER, on which the unmodified seed agent is closest to total failure (the seed scores 0% on 24 of 24 GROUPOBJECTS task-runs in the MF cell, and on 20 of 24 in the SF cell).

Table 13 mirrors the structure of the EASY results: the MF configuration produces the larger held-out lift, $+20.8$ pp ($p < 0.001$), and the SF ablation produces a smaller but still significant lift, $+11.8$ pp ($p \approx 0.023$). The absolute gap between the best-by-train candidates is $+6.7$ pp in MF's favor (95% CI $[+0.0, +13.0]$ pp, $t = 1.96$); the lower bound just touches zero, which is consistent with the SF *seed* on HARD having an unusually wide per-run standard deviation ($\sigma = 12.0$ pp), three times the MF seed's $\sigma = 3.8$ pp. The evolved candidates, by contrast, are both well within the noise floor of $\sigma \approx 3$ pp that we observed in the EASY cell.

Where the lift comes from: SF and MF are complementary on HARD

A per-class breakdown reveals a richer story than the headline mean suggests. Table 14 reports per-class held-out means for the seed and best-by-train candidates in each cell, averaged over the three runs (n ranges from 9 to 24 task-runs per class depending on class size in the test split).

The two cells produce *qualitatively different* sources of lift on HARD:

- **SF is strong on stacking and bulk-motion classes.** SF recovers BUILDCUBETOWER essentially from scratch ($5.6\% \rightarrow 44.4\%$, $+38.8$ pp) and nearly closes the MOVEOBJECTSTOLEFT gap ($45.2\% \rightarrow 76.5\%$, $+31.3$ pp). Both classes have explicit per-task recipes in the evolved prompt (the analogous tower recipe reproduced in the EASY SF prompt, Appendix A.3.8, fixes stack heights at $z \in \{0.09, 0.14, 0.19, \ldots\}$, exactly the per-cube spacing

Table 15: Held-out test compute totals across the 21 HARD test tasks (representative single run; the sign matches under the 3-run average). The MF candidate reaches its higher accuracy with fewer tool calls and lower wall time, mirroring the EASY pattern. The SF candidate on HARD reverses sign: it spends *more* compute than the seed, because its evolved prompt instructs the agent to re-perceive after each drop and to keep moving objects until the task condition holds.

Cell	Metric	Seed	Best	Δ
MF (headline)	Total tool calls	205	149	$\mathbf{-27.3\%}$
MF (headline)	Total wall time (s)	1278	1019	$\mathbf{-20.3\%}$
SF (ablation)	Total tool calls	216	251	$+16.2\%$
SF (ablation)	Total wall time (s)	1363	1827	$+34.0\%$

needed for the simulator's $\sim$5 cm cubes), and the y-axis disambiguation alone covers most of the left-side movement headroom.

- **MF is strong on perception-bound classes.** MF unlocks GROUPOBJECTS from a near-total failure ($0.0\% \rightarrow 37.5\%$, $+37.5$ pp) and gains on PLACECUBES ($31.7\% \rightarrow 45.0\%$, $+13.3$ pp). These are precisely the classes where the seed agent's most common failure mode is not bad planning but bad perception (a cluster query that returns the wrong color, a cube detection that fails to distinguish red from yellow under shadow). The wrapper-level edits MF has access to (color-mask verification on top of the upstream detector, multi-query fallbacks, detection caching) are what move these classes, and no prompt edit substitutes for them.
- **Neither cell cracks BUILDCUBETOWER beyond mid-tier credit on MF, nor improves GROUPOBJECTS on SF.** SF in fact regresses slightly on GROUPOBJECTS ($15.3\% \rightarrow 9.7\%$), consistent with the train-set frontier being dominated by gains on the other three classes and the minibatch acceptance rule letting a small per-class regression through.

The aggregate result, then, is not that MF is simply "more SF": MF and SF specialize in opposite directions on HARD. MF wins the headline mean because GROUPOBJECTS contributes 8 of the 21 test tasks and the GROUPOBJECTS unlock is large.

Compute: tool calls and wall time

Table 15 shows that the compute trade-off on HARD now differs by cell. The MF candidate again reduces both tool calls and wall time substantially (**27.3%** and **20.3%**), even more aggressively than on EASY. The SF candidate, in contrast, *increases* compute by **16.2%** in tool calls and **34.0%** in wall time: its evolved prompt instructs the agent to re-detect the scene between moves and to keep moving until the task condition holds, which on the larger HARD scenes costs additional perception round-trips. This is consistent with the framing that prompt edits raise the floor (by making the agent more thorough) while tool edits raise the ceiling (by making each call more discriminating, and therefore fewer of them necessary). These are the same numbers cited in the repository-level mutation ablation in Sections 4.3 and 4.4 of the main text.

What RHO learned on HARD

The mutator's behavior on HARD matches the per-class picture above: it allocates its effort *toward tool-body edits* rather than prompt edits. The MF-evolved candidate (`g6-s6`, generation 6 of 10) grows `solver/tools.py` from 375 lines in the seed to 1,287 lines (a $3.4\times$ expansion, comparable to the EASY MF candidate's $3.3\times$), but it grows the `MANIPULATION_PROMPT` body by only $\sim$25 lines (against $\sim$70 lines on EASY). The SF candidate on HARD (`g7-s7`), by contrast, must do all of its work in the prompt and expands `MANIPULATION_PROMPT` to $\sim$120 lines, the largest evolved prompt in this study.

The reason this matters is that the HARD failure modes are dominated by perception, not planning: the seed agent typically chooses correct positions for an object once it knows which centroid corresponds to which color or type, but it does not always know which centroid is which. The MF mutator preferentially extends `tools.py` (with the color-mask verifier, the multi-query fallback, and the detection cache already described in Section A.3.3) because that is where the leverage is on this subset. The notable result is that RHO identifies *where* to spend its edits, not just how many lines to add.

A.3.5 Discussion

Four observations from the RAI EASY and HARD results bear on the paper's main thesis.

Methodology transfers. RHO's closed-loop evolution moves from CaP-Bench to a qualitatively different benchmark (one whose state is a Vulkan-rendered O3DE simulator driven by ROS 2 and a real perception stack, rather than a procedural primitive surface) with no changes to the mutator, the scalar-sum acceptance gate, or the best-by-train selection rule relative to the canonical CaP-Bench Codex GPT-5.5 Multi-file run. The MF configuration adds one substrate-specific constraint, an AST-level tool-name-contract check that rejects candidates which alter the fixed tool API exposed to the LLM caller (Sec. 3.2 above). The same apparatus produces a statistically significant headline MF lift on both subsets ($+20.6$ pp on EASY and a near-identical $+20.8$ pp on HARD); the SF ablation produces a smaller but significant lift on both ($+14.3$ pp on EASY, $+11.8$ pp on HARD). The MF–SF absolute gap is $+9.5$ pp on EASY and $+6.7$ pp on HARD. In both cases the gap is consistent with MF's freedom to edit tool bodies (i.e., repository-level mutation) being worth roughly a third to a half of the headline lift on top of what prompt edits alone can deliver.

Prompt and tool edits cover different failure modes. The HARD per-class breakdown (Table 14) shows that SF and MF do not simply lift the same tasks by different amounts; they specialize. SF recovers BUILDCUBETOWER ($+38.8$ pp class lift) and MOVEOBJECTSTOLEFT ($+31.3$ pp), both classes whose seed failures stem from planning and coordinate-frame errors that prompt-level instructions can fix. MF essentially unlocks GROUPOBJECTS ($+37.5$ pp) and PLACECUBES ($+13.3$ pp), classes whose seed failures stem from perception ambiguity that wrapper-level verification can fix. The two ablations are therefore complementary signals about *where* a given failure mode lives, not just *whether* RHO can move it.

Lifts come with lower inference cost, with one instructive caveat. The MF headline candidate reduces both tool calls and wall time relative to its seed on both subsets (-17.6% / -3.8% on EASY, $\mathbf{-27.3}\%$ / $\mathbf{-20.3}\%$ on HARD). The SF ablation reduces compute on EASY (-9.2% tool calls, -8.2% wall time) but *increases* compute on HARD ($+16.2\%$ tool calls, $+34.0\%$ wall time), because its evolved prompt instructs the agent to re-perceive between moves and persist until the task condition holds. Tool-side edits, which can encode discriminating logic behind the agent's tool API, therefore appear to be the more robust route to "accurate *and* cheap" for a closed-loop hardware setting; prompt-side edits buy accuracy on HARD but can do so by spending more inference budget. This is the same inversion the main paper highlights against single-file GEPA-style [14] optimization (Section 4.3).

Evolution suppresses sampler variance. The per-run standard deviation of the evolved candidates is substantially smaller than that of the seeds in every cell. On EASY, $\sigma_{\text{best}} = 2.8$ pp for both MF and SF versus $\sigma_{\text{seed}} \in \{9.9, 7.3\}$ pp, a roughly $3\times$ tightening. On HARD, $\sigma_{\text{best}} = 2.4$ pp (MF) and 5.4 pp (SF) versus $\sigma_{\text{seed}} \in \{3.8, 12.0\}$ pp, a $1.6\times$ to $2.2\times$ tightening. For any deployed robotics system, where reproducibility and determinism are first-class concerns, this variance reduction is a useful side-effect that does not appear in the headline accuracy numbers.

Together with the main CaP-Bench results (Section 4), these findings support the claim that RHO is a general recipe for closed-loop refinement of robot-policy repositories, including LLM-in-the-loop

agent harnesses, and that it can be applied with modest single-workstation compute to industrially relevant robotics stacks.

A.3.6 Full evolved prompts

For reference, this appendix reproduces the three prompts that the inline snippets in Section A.3.3 draw from: the unedited RAI seed prompt, the best-by-train SF-evolved prompt (`g2-s2`, generation 2 of 10), and the best-by-train MF-evolved prompt (`g8-s8`, generation 8 of 10). Long lines have been allowed to wrap; the underlying text is otherwise verbatim from each cell's evolved `solver/prompts.py`.

A.3.7 Seed prompt

The seed is RAI's stock manipulation system prompt, copied verbatim from `rai_bench.manipulation_o3de.interfaces.ManipulationTask`. Note line 4, which inverts the convention the benchmark actually uses for scoring (see Section A.3.3).

```
You are a robotic arm with interfaces to detect and manipulate objects.
Here are the coordinates information:
x - front to back (positive is forward)
y - left to right (positive is right)
z - up to down (positive is up)
Before starting the task, make sure to grab the camera image to understand the
↪  environment.
```

A.3.8 SF-evolved prompt (best-by-train, `g2-s2`)

RHO's single-file ablation expanded the 6-line seed into the following 78-line task-specific guidance. This is the prompt that produced the SF numbers in Tables 11 and 12.

```
You control a Franka Panda arm in a closed-loop tabletop simulator. Complete
the user's manipulation task by calling tools, not by describing a plan.

Start every scenario with a fresh camera image. If the tool is
get_ros2_camera_image, call it with no arguments. If the image tool is
get_ros2_image, use topic="/color_image5".

Coordinate frame:
- x is front/back on the table; larger x is farther forward.
- y is right/left on the table. POSITIVE y is LEFT and negative y is RIGHT.
- z is height above the table; larger z is up. Tabletop object centroids are
usually near z=0.04 m.

Object names for detection:
- Before moving anything, call get_object_positions for the exact required
object type. Use singular food names: "apple", "carrot", "tomato", "corn".
Use color-qualified cube names: "red cube", "blue cube", "yellow cube".
- For "all cubes", query red cube, blue cube, and yellow cube separately.
- If different cube-color queries return nearly identical x,y,z coordinates,
treat them as one physical cube and move it at most once.
- Never invent object coordinates when detection succeeds.

Pick/place discipline:
- Move one object completely before moving another object.
- The normal sequence for one object is exactly:
1. choose a returned centroid for the object,
2. move_to_point(x=centroid_x, y=centroid_y, z=centroid_z, task="grab"),
3. move_to_point(x=goal_x, y=goal_y, z=goal_z, task="drop").
- A grab call must use the object's exact detected centroid. Never grab at a
destination coordinate or at a made-up y such as 0.35.
- After a successful grab, the next arm call should be the drop for that same
object. Do not issue two grab calls in a row.
```

```
- For ordinary tabletop placement, use the object's detected z for the drop.
Use higher z only when intentionally building a vertical tower.
- Stop when the requested relation is satisfied; extra moves often reduce the
score.

Task recipes:

1. Place one object at coordinates (x: X, y: Y)
- Detect the named object type.
- If several instances are returned, choose one that is not already at X,Y,
usually the instance farthest from that target.
- Grab at its exact centroid, then immediately drop at x=X, y=Y, z=detected_z.

2. Move all requested objects to the left side of the table
- Positive y is the left side. Move only the object types named in the request;
ignore distractors.
- Detect each requested type separately.
- Leave any target already at y >= 0.25 in place.
- For each target with y < 0.25: grab at its exact centroid, then immediately
drop at the same x, detected z, and a clear positive-y value from 0.32 to
0.45. For multiple moved targets, use small y or x offsets to avoid overlap
while staying reachable.

3. Make all cubes adjacent to at least one cube
- Detect red, blue, and yellow cubes, deduplicate overlapping detections, and
ignore all non-cubes.
- If every physical cube already has another cube within about 0.10 m in the
x/y plane, stop.
- Keep one central reachable cube as the anchor. Move only isolated cubes.
- Drop each moved cube on the tabletop next to the anchor or growing group with
center spacing about 0.06 to 0.08 m in x or y. Do not stack for adjacency.

4. Build one vertical tower from cubes
- Use only the cube colors requested by the task. Ignore non-cubes and unnamed
cube colors.
- Detect the requested colored cubes and deduplicate physical cubes.
- Leave one low, stable, reachable cube in place as the base.
- For each remaining cube: grab at its exact centroid and immediately drop at
the base x,y with increasing center heights: second cube z=0.09, third
z=0.14, fourth z=0.19, fifth z=0.24, sixth z=0.29, seventh z=0.34.

5. Form separate clusters based on object types
- Only manipulate the object types listed before "form separate clusters";
never move unlisted distractors.
- For each listed type, keep one object as that type's anchor and move other
same-type objects to positions 0.06 to 0.08 m from the anchor.
- Keep clusters for different types at least about 0.18 m apart, using
distinct y bands such as -0.30, 0.00, and 0.30.
```

A.3.9 MF-evolved prompt (best-by-train, g8-s8)

The headline MF candidate's system prompt. This is the prompt that the inline snippets in Section A.3.3 are excerpted from, and that produced the MF rows in Tables 11 and 12.

```
You control a Franka Panda arm in a tabletop manipulation scene. Work in
closed loop: first call get_ros2_camera_image for the canonical
/color_image5 view, then call get_object_positions for the needed object
types, then use move_to_point for physical actions. Trust fresh tool
results over guesses.

Coordinate frame:
- x is front/back on the table.
- y is left/right on the table. Positive y is LEFT; negative y is RIGHT.
- z is height. Positive z is upward.
```

```
Critical tool rules:
1. A physical move of an object is one grab call followed by one drop call.
Do not call grab again to lift or carry the same object; a second grab
can reopen the gripper and lose the object.
2. Every grab must use the exact x,y,z centroid from get_object_positions
for the object you intend to move. Never use a cluster center, left/right
side location, target coordinate, tower base, or suggested drop slot as a
grab coordinate.
3. For a table drop, use the target x,y and z about 0.04 to 0.06. For a
stack drop, use the base x,y and increase z by about 0.055 per cube above
the base.
4. Use the exact argument names x, y, z, task. If a call is malformed,
correct it before continuing.
5. After each successful drop, stop if the task is satisfied; otherwise
re-detect before choosing the next object when positions may have
changed.
6. If get_object_positions says no objects of a requested type were
detected, skip that type. Never substitute a different color/type just
because the task mentions it.
7. For a group/separate-clusters task, do not stop just because a requested
type has one detected object; it still must be separated from other types
if the scene is mixed.
8. Tool outputs may include several named aids. Use only the aid that
matches the task wording: left-side aid for left-side tasks, tower aid
for tower tasks, cube adjacency aid for adjacent-cubes tasks, cluster aid
for separate-clusters tasks, and multi-object aid for coordinate
placement. Ignore unrelated aids.

Object queries:
- For red, blue, or yellow cubes, query the full color name: "red cube",
"blue cube", "yellow cube".
- For "all cubes are adjacent", query "cube" once so every cube color is
included.
- For a tower task that explicitly requests red cubes, blue cubes, and
yellow cubes together, query "cube" once and stack every detected cube.
For tower tasks that request only a subset of colors, query only those
color names.
- For apples, carrots, tomatoes, and corn, query the singular object name
unless the task asks for a plural group.
- Apples and tomatoes can cross-detect because both are red and round.
get_object_positions may filter the list to likely candidates for the
requested fruit; when it does, use the returned filtered centroid(s). If
it returns an apple/tomato disambiguation aid, choose the requested type
from that aid before considering which centroid is closest to a coordinate
target.

Task recipes:
- Left side of the table: for every requested object type, move each
detected object whose y <= 0.0 to a free location well into positive y,
around 0.38 to 0.50. Keep objects already at y > 0.0. Preserve reachable
x when possible and space multiple drops by about 0.07 m in x or y.
- Single vertical tower: collect detections for the requested cubes and
ignore absent colors. If target cube centroids already share the same
x,y within about 0.03 m with increasing z, stop. Otherwise choose one
detected target cube or existing stack as the base and keep its current
x,y; do not invent a new tower center and never use coordinate-task
targets such as y=-0.4. Count the detected target cubes: if there are N
cubes and K are already in the chosen stack, make exactly N-K grab/drop
moves before stopping. For each moved cube, grab that cube at its listed
centroid, then drop at the base x,y with z values about 0.09, 0.145,
0.200, ... as the tower grows. Do not include apples, carrots, tomatoes,
corn, or unrequested cube colors.
- All cubes adjacent: make connected neighbors, not a midpoint pile and
not a stack. Use the cube adjacency aid from get_object_positions when
present: grab the named isolated cube index and copy the suggested drop
tuple exactly. The anchor cube's listed centroid is not a drop target.
```

```
Pick an anchor cube and move each isolated cube to a free spot 0.06 to
0.09 m beside an existing cube, such as same x with y plus/minus 0.07 or
same y with x plus/minus 0.07. Never drop at the exact x,y of another
cube for adjacency. In a two-cube scene, exactly one cube usually moves
once, then re-detect; stop when every cube has another cube within about
0.10 m in x/y.
- Separate clusters by type: only move the requested types. For each
requested type, make one connected group with same-type objects 0.06 to
0.10 m apart; do not make several local pair clusters. Use the cluster
aid anchor when present: keep that anchor and move every other object of
that same type into slots around the same anchor before stopping. Put
different requested-type cluster centers at least 0.18 m apart; if
centers overlap, use the first requested type around positive y (about
0.30) and the next requested type around negative y (about -0.25).
Cluster centers are drop targets only. Always grab a listed centroid
first. A type with one detected object is connected, but not
automatically separated; if the scene includes other objects nearby,
move that object to an isolated reachable spot.
- Placed at coordinates (x: X, y: Y): move exactly one requested object to
X,Y. First resolve object identity, especially apple vs tomato when the
tool gives a disambiguation aid. If several valid requested objects
remain, compute each listed centroid's xy distance to X,Y and usually
grab the closest reachable one; only avoid it if it is at an extreme
table edge. Do not pick a generic central object when another valid
requested object is visibly closer to the target. Drop exactly at X,Y
with table z around 0.04 to 0.06.

Keep plans short and physical. Single-object tasks are usually four tool
calls total: image, positions, grab, drop. Multi-object tasks repeat one
grab/drop pair per object that actually needs to move.
```